TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN HỌC PHẦN
NHẬP MÔN DỮ LIỆU LỚN

ĐỀ TÀI

TRÌNH BÀY KẾT QUẢ PHÂN TÍCH DỮ LIỆU THƯƠNG MẠI ĐIỆN TỬ OLIST BẰNG NEON SERVERLESS POSTGRES VÀ APACHE SUPERSET

Giai đoạn Analytics → Presentation trong pipeline dữ liệu lớn

Quy trình phụ trách: **Analytics → Presentation**

Lớp: DCT123C5

Giảng viên hướng dẫn: Vũ Ngọc Thanh Sang

STT	Họ và tên	MSSV	Vai trò chính
1	Võ Việt Tường	3123411334	Nhóm trưởng – kiến trúc, Neon & loader, tích hợp
2	Nguyễn Hoàng Phi Long	3123411180	Semantic layer SQL & dashboard Superset
3	Mai Hồng Phong	3123411225	Thực nghiệm, minh chứng & báo cáo

Thành phố Hồ Chí Minh, năm 2026

Tóm tắt

Đồ án thực hiện giai đoạn **Analytics → Presentation** của pipeline dữ liệu lớn trên bộ dữ liệu thương mại điện tử Olist (Brazil, khoảng 100 nghìn đơn hàng 2016–2018, 1,55 triệu dòng thô). Đầu vào là 16 bảng Parquet (194.132 dòng) do nhóm Analytics tính trên cụm Apache Spark, bao phủ sáu chủ đề: xu hướng KPI, danh mục, địa lý – logistics, phân khúc khách hàng RFM/K-Means, người bán, giao trễ – đánh giá – thanh toán. Nhóm xây dựng tầng phục vụ gồm: loader Python (pyarrow + COPY) nạp dữ liệu vào **Neon Serverless Postgres** (tách compute/storage, scale-to-zero), kèm 7 bảng gold để lọc chéo; tầng ngữ nghĩa 22 view SQL chuẩn hoá KPI; dashboard **Apache Superset** (Docker) 7 tab, 45 chart, 7 bộ lọc native và khu vực gợi ý quyết định (7 luật có ngưỡng, 153 người bán cần theo dõi) cho giai đoạn Decision/Action. Kết quả: 505.592 dòng (23 bảng) nạp trong 351,34 giây, 23/23 bảng khớp số dòng, 19/19 kiểm tra số neo đạt (doanh thu 13.494.400,74 BRL; đỉnh 11/2017 tăng 52,06%; đơn trễ 2,56 sao so với 4,29; 3,04% khách mua lại). Truy vấn dashboard có trung vị 173,4 ms (server 1,25 ms); một tab mở trong 1,04 giây khi có cache; thí nghiệm ×10 cho thấy view tổng hợp giữ 1,1 ms trong khi truy vấn dữ liệu thô tăng lên 506,6 ms. Hệ thống tái lập được bằng Docker Compose và mã nguồn công khai.

Từ khoá: Big Data, Parquet, Apache Spark, serving layer, Neon, serverless PostgreSQL, semantic layer, Apache Superset, dashboard, Olist.

Thông tin nhóm

Hạng mục	Thông tin
Học phần / lớp	Nhập môn Dữ liệu lớn – DCT123C5, Trường Đại học Sài Gòn
Giảng viên	Vũ Ngọc Thanh Sang
Thành viên	Võ Việt Tường (3123411334, nhóm trưởng); Nguyễn Hoàng Phi Long (3123411180); Mai Hồng Phong (3123411225)
Quy trình phụ trách	Analytics → Presentation
Đầu vào	16 bảng Parquet của nhóm Analytics + 7 bảng gold, tài liệu `ANALYTICS_HANDOVER.md`
Đầu ra cho giai đoạn sau	View `presentation.v_decision_hints`, `v_seller_watchlist` trên Neon; dashboard Superset
Kho mã nguồn	https://github.com/viettuong287/BigDataProject

Phân công thực tế và mức đóng góp của từng thành viên ở Phụ lục G.

MỤC LỤC

Tóm tắt i

Thông tin nhóm ii

Danh mục từ viết tắt ix

1. Giới thiệu 1

1.1. Bối cảnh 1

1.2. Quy trình được lựa chọn và ranh giới 1

1.3. Mục tiêu và phạm vi 2

1.4. Câu hỏi kỹ thuật 2

1.5. Cấu trúc báo cáo 3

2. Cơ sở và công nghệ liên quan 4

2.1. Đặc trưng dữ liệu lớn và vai trò tầng trình bày 4

2.2. Data Lake và định dạng cột Apache Parquet 4

2.3. Xử lý phân tán bằng Apache Spark ở giai đoạn trước 4

2.4. OLAP, bảng tổng hợp và tầng phục vụ 5

2.5. PostgreSQL và Neon Serverless Postgres 6

2.6. Tầng ngữ nghĩa và Apache Superset 6

2.7. Nguyên tắc thiết kế dashboard 7

2.8. Các phương án công nghệ thay thế 7

3. Dữ liệu và yêu cầu 8

3.1. Nguồn dữ liệu và dòng dõi 8

3.2. Đặc trưng Big Data của bài toán 8

3.3. Input 9

3.4. Output 10

3.5. Data contract giữa Analytics và Presentation 10

3.6. Yêu cầu chức năng và phi chức năng 11

3.7. Định nghĩa KPI 12

4. Thiết kế hệ thống 13
4.1. Kiến trúc tổng thể 13
4.2. Luồng dữ liệu, định dạng và giao thức 14
4.3. Thiết kế tầng lưu trữ trên Neon 15
4.4. Thiết kế tầng ngữ nghĩa 16
4.5. Thiết kế dashboard và giao diện với Decision/Action 16
4.6. Lựa chọn công nghệ và lý do 18
5. Cài đặt và triển khai 19
5.1. Môi trường và cấu trúc mã nguồn 19
5.2. Khởi tạo Neon và nạp dữ liệu 19
5.3. Triển khai Apache Superset 20
5.4. Xử lý lỗi và các vấn đề kỹ thuật 20
6. Thực nghiệm và đánh giá 22
6.1. Kịch bản, môi trường và tiêu chí 22
6.2. E1 – Đối chiếu dữ liệu 22
6.3. E2 – Hiệu năng nạp dữ liệu 23
6.4. E3 – Độ trễ truy vấn 23
6.5. E4, E5 – Thời gian tải dashboard và bộ lọc 24
6.6. E6 – Thí nghiệm mở rộng 24
6.7. Đánh giá tổng hợp 25
7. Kết quả demo 26
7.1. Luồng demo từ đầu vào đến đầu ra 26
7.2. Dashboard và nội dung phân tích 27
7.3. Cảnh báo người bán và gợi ý quyết định 29
7.4. Đầu ra cho giai đoạn Decision/Action 29
8. Thảo luận 31
8.1. Điểm mạnh 31
8.2. Hạn chế 31

8.3. Khả năng mở rộng 32
8.4. Bài học kỹ thuật và hướng cải tiến 33
9. Kết luận 34
Tài liệu tham khảo 36
Phụ lục A. Hướng dẫn chạy 38
Phụ lục B. So sánh phương án công nghệ 39
Phụ lục C. Data contract chi tiết 40
Phụ lục D. Danh mục đối tượng tầng ngữ nghĩa 42
Phụ lục E. Minh chứng thực nghiệm chi tiết 44
E.1. Kết quả nạp từng bảng 44
E.2. Kiểm tra số neo và nhất quán 44
E.3. Độ trễ 22 truy vấn dashboard 45
E.4. Ảnh chụp bổ sung 46
E.5. Danh sách tệp minh chứng 47
Phụ lục F. Mã nguồn và cấu hình 49
Phụ lục G. Bảng phân công và đóng góp 51
Phụ lục H. Ghi chú sử dụng công cụ AI 52

DANH MỤC HÌNH

Hình 2.1. Spark Master UI (cổng 8080) của nhóm Analytics: 2 worker ALIVE, 4 core, ứng dụng olist-analytics FINISHED 5

Hình 4.1. Sơ đồ data-flow của hệ thống: Parquet → Loader (COPY, TLS 5432 direct) → Neon (gold, analytics → presentation) → Superset (SQLAlchemy, endpoint pooled) → người dùng; nhánh v_decision_hints → Decision/Action 14

Hình 7.1. Tab 1 "Tổng quan & xu hướng": 6 thẻ KPI khung 2017-01 → 2018-08, doanh thu tháng gần nhất, doanh thu & số đơn theo tháng, % trễ vs review, MoM 27

Hình 7.2. Tab 3 "Địa lý & logistics": bản đồ doanh thu theo bang và quan hệ ngày giao – review (kích thước = doanh thu) 28

Hình 7.3. Bộ lọc native: khoảng thời gian 2018 + bang SP — thẻ KPI và biểu đồ tháng chỉ còn 8 tháng 2018 của bang SP (doanh thu 2.960.326 BRL, 23.598 đơn, trễ 6,82%) 28

Hình 7.4. Tab 5: bảng watchlist cảnh báo 153 người bán (số vi phạm, lý do, mức ĐỎ/VÀNG, hành động đề xuất) 29

Hình 7.5. Tab 7: bảng "Gợi ý quyết định" — luật, giá trị hiện tại, ngưỡng và hành động (nguồn v_decision_hints) 29

Hình E.1. Tab 2 "Danh mục & mùa vụ": Pareto top 25, heatmap mùa vụ 2017, tỷ lệ ship/doanh thu 46

Hình E.2. Tab 4 "Khách hàng & RFM": thẻ khách hàng, phân khúc RFM, cụm K-Means 46

Hình E.3. Kiểm kê Neon sinh từ catalog database: 25 bảng, 22 view, số dòng, dung lượng, 0,25 CU, PostgreSQL 18 47

DANH MỤC BẢNG

Bảng 3.1. Dòng dõi dữ liệu từ nguồn đến tầng phục vụ 8

Bảng 3.2. Các thành phần đầu ra của giai đoạn Presentation 10

Bảng 3.3. Data contract Analytics → Presentation: danh sách bảng, grain và khoá 10

Bảng 3.4. Định nghĩa KPI và quy tắc tổng hợp trên dashboard 12

Bảng 4.1. Các bước của luồng dữ liệu 14

Bảng 4.2. Cấu hình tầng phục vụ Neon (theo `evidence/neon_project.json`) 15

Bảng 4.3. Thiết kế dashboard theo tab và nhóm người dùng 16

Bảng 4.4. Bảy luật gợi ý quyết định trong v_decision_hints 17

Bảng 4.5. Công nghệ sử dụng, vai trò và lý do lựa chọn 18

Bảng 5.1. Môi trường cài đặt và triển khai 19

Bảng 5.2. Các vấn đề kỹ thuật đã gặp và cách xử lý 20

Bảng 6.1. Các kịch bản thực nghiệm 22

Bảng 6.2. Tóm tắt độ trễ truy vấn (`query_latency.md`, `superset_chart_latency.csv`) 23

Bảng 6.3. Thời gian hiển thị từng tab dashboard: cold và có cache (`evidence/dashboard_load_time.csv`) 24

Bảng 6.4. Kiểm thử bộ lọc native (ảnh `evidence/filter_*.png`, thời gian từ `dashboard_load_time.csv`) 24

Bảng 6.5. Thí nghiệm mở rộng trên Neon 0,25 CU: nhân bản `gold.fact_order_items` ×N (`evidence/scale_experiment.csv`) 24

Bảng 6.6. Đánh giá mức đạt theo tiêu chí 25

Bảng B.1. So sánh công cụ trình bày (BI) cho giai đoạn Presentation 39

Bảng B.2. So sánh phương án tầng lưu trữ phục vụ 39

Bảng C.1. Ánh xạ kiểu dữ liệu Parquet → PostgreSQL do loader áp dụng 40

Bảng C.2. Các bảng gold được nạp bổ sung vào schema gold (đầu vào chi tiết cho bộ lọc chéo) 40

Bảng C.3. Schema 16 bảng Analytics trên Neon (sinh tự động từ Parquet) 40

Bảng D.1. Các đối tượng của tầng ngữ nghĩa: 22 view và 1 bảng dimension (quét tự động từ `sql/10_presentation_views.sql`) 42

Bảng E.1. Kết quả nạp từng bảng, lần chạy `20261112-145508-40efaf` (tệp `evidence/load_log.csv`) 44

Bảng E.2. Đối chiếu số neo bàn giao của Analytics và tính nhất quán giữa các view trên Neon 44

Bảng E.3. Độ trễ truy vấn các view trên Neon (22/22 phép đo thành công; tệp `evidence/query_latency.csv`) 45

Bảng E.4. Danh sách tệp minh chứng trong evidence/ 47

Bảng G.1. Bảng phân công nhiệm vụ và mức đóng góp của các thành viên 51

Bảng G.2. Ma trận đóng góp theo sản phẩm bàn giao (C = chính, H = hỗ trợ) 51

Danh mục từ viết tắt

Viết tắt	Diễn giải
AOV	Average Order Value – giá trị đơn hàng trung bình
BI	Business Intelligence
BRL	Real Brazil (đơn vị tiền tệ)
CU	Compute Unit của Neon (≈ 1 vCPU + 4 GB RAM)
GMV	Gross Merchandise Value = doanh thu hàng + phí vận chuyển
KPI	Key Performance Indicator
MoM	Month-over-Month
OLAP / OLTP	Online Analytical / Transaction Processing
PTTT	Phương thức thanh toán
RFM	Recency – Frequency – Monetary
RTT	Round-trip time – thời gian khứ hồi mạng
TLS	Transport Layer Security

1. Giới thiệu

1.1. Bối cảnh

Trong hệ thống dữ liệu lớn, giá trị kinh doanh chỉ xuất hiện khi kết quả phân tích đến được người ra quyết định, đúng lúc và dễ hiểu. Các kiến trúc tham chiếu như Lambda [8] hay Lakehouse [9] đều có một **tầng phục vụ (serving layer)**: dữ liệu sau khi được xử lý phân tán được cô đọng thành bảng tổng hợp, nạp vào hệ thống trả lời truy vấn tương tác với độ trễ thấp, rồi được trình bày qua dashboard hoặc API. Thiếu tầng này, kết quả Spark chỉ là các tệp Parquet trong Data Lake mà người dùng nghiệp vụ không khai thác được.

Bộ dữ liệu dùng xuyên suốt pipeline của lớp là **Brazilian E-Commerce Public Dataset by Olist** [1]: khoảng 100 nghìn đơn hàng trên các sàn thương mại điện tử Brazil từ 09/2016 đến 10/2018. Dữ liệu đã qua các giai đoạn trước: nhóm Storage → Processing dựng tầng gold (7 bảng, đối chiếu 11/11); nhóm Processing → Analytics chạy 14 truy vấn Spark SQL và K-Means trên cụm Spark, bàn giao 16 bảng Parquet kèm tài liệu `ANALYTICS_HANDOVER.md`. Nhóm chúng tôi đưa các kết quả này tới người dùng nghiệp vụ của sàn: ban điều hành, quản lý ngành hàng, vận hành – logistics, CRM – marketing và đội quản lý người bán.

1.2. Quy trình được lựa chọn và ranh giới

Nhóm thực hiện quy trình con **Analytics → Presentation** (Mục 4 của đề bài), với ranh giới:

- **Điểm bắt đầu:** 16 bảng Parquet trong `data/analytics/` cùng tài liệu bàn giao của Analytics; nạp kèm 7 bảng gold (311.460 dòng, cũng là đầu vào của Analytics) để dashboard lọc chéo được ở cấp đơn hàng. Nhóm **không** đọc lại CSV thô, **không** chạy lại Spark và **không** tính lại các phân tích phức tạp (K-Means, Pareto, RFM). Mọi chỉ số trên dashboard truy ngược được về một cột của Analytics hoặc gold, và tổng từ gold phải khớp số của Analytics.
- **Điểm kết thúc:** (1) cơ sở dữ liệu phục vụ trên Neon đã nạp và kiểm tra; (2) tầng ngữ nghĩa gồm các view SQL định nghĩa KPI thống nhất; (3) dashboard Superset có KPI, bộ lọc, biểu đồ và nguồn dữ liệu rõ ràng; (4) các view **gợi ý quyết định** cho giai đoạn Presentation → Decision/Action.

- **Giao diện với giai đoạn trước:** *data contract* gồm danh sách bảng, grain, khoá, kiểu dữ liệu và các số neo đối chiếu (Chương 3). **Với giai đoạn sau:** các view ổn định trong schema `presentation`, đọc bằng SQL, không phụ thuộc Superset.

1.3. Mục tiêu và phạm vi

Mục tiêu là xây dựng tầng trình bày **đúng số liệu, tương tác được, tái lập được và mở rộng được**:

1. Nạp đầy đủ, chính xác các bảng Parquet vào cơ sở dữ liệu phục vụ trên đám mây, bảo toàn kiểu dữ liệu (đặc biệt tiền tệ `DECIMAL`), có nhật ký nạp và kiểm tra đối chiếu tự động.
2. Xây dựng tầng ngữ nghĩa trong đó mỗi KPI có một định nghĩa và cách tổng hợp đúng (kể cả chỉ số không cộng dồn như tỷ lệ, trung bình).
3. Triển khai dashboard Superset cho năm nhóm người dùng, có bộ lọc native và khu vực gợi ý quyết định.
4. Đo thời gian nạp, thông lượng, độ trễ truy vấn và tác động của scale-to-zero, bộ nhớ đệm.
5. Đóng gói bằng Docker Compose và mã nguồn để người khác chạy lại.

Ngoài phạm vi: thu thập, làm sạch, tính toán phân tích (các nhóm trước); thực thi hành động tự động như gửi email, gọi webhook (nhóm Decision/Action). Dữ liệu được xử lý theo lô vì nguồn Olist là dữ liệu lịch sử.

1.4. Câu hỏi kỹ thuật

- **RQ1 – Lưu trữ phục vụ:** phục vụ kết quả từ đâu (Parquet trực tiếp, Postgres cục bộ, ClickHouse hay Postgres serverless) để cân bằng độ trễ, chi phí, khả năng chia sẻ và công vận hành? (Chương 2, 4)
- **RQ2 – Tính đúng đắn:** làm sao đảm bảo số trên dashboard khớp tuyệt đối kết quả Spark, kể cả khi người dùng lọc và gộp chỉ số không cộng dồn? (Chương 3, 6)
- **RQ3 – Hiệu năng:** thời gian nạp và độ trễ truy vấn là bao nhiêu; nút thắt nằm ở đâu? (Chương 6)
- **RQ4 – Thiết kế trình bày:** bố cục, biểu đồ, bộ lọc nào giúp từng nhóm người dùng trả lời câu hỏi nhanh nhất? (Chương 4, 7)

- **RQ5 – Mở rộng và tích hợp:** kiến trúc thay đổi ra sao khi dữ liệu tăng 10–100 lần, và giai đoạn sau tiêu thụ kết quả thế nào? (Chương 8, 9)

1.5. Cấu trúc báo cáo

Chương 2 trình bày cơ sở và công nghệ; Chương 3 mô tả dữ liệu, đặc trưng Big Data, đầu vào, đầu ra và data contract; Chương 4 trình bày kiến trúc và thiết kế; Chương 5 mô tả cài đặt; Chương 6 thực nghiệm; Chương 7 kết quả demo; Chương 8 thảo luận; Chương 9 kết luận. Phụ lục chứa hướng dẫn chạy, cấu hình, schema, mã SQL, minh chứng chi tiết, mã nguồn, bảng phân công và ghi chú sử dụng AI.

2. Cơ sở và công nghệ liên quan

Chương này chỉ trình bày phần kiến thức Big Data dùng trực tiếp trong đồ án.

2.1. Đặc trưng dữ liệu lớn và vai trò tầng trình bày

Dữ liệu lớn được mô tả bằng các chữ V: **Volume** (khối lượng), **Velocity** (tốc độ phát sinh), **Variety** (đa dạng), **Veracity** (độ tin cậy) và **Value** (giá trị). Pipeline gồm Sources → Ingestion → Storage → Processing → Analytics → Presentation → Decision/Action; mỗi giai đoạn giao tiếp qua định dạng dữ liệu thống nhất. Đặc điểm quan trọng là **khối lượng giảm dần theo từng tầng trong khi giá trị trên mỗi byte tăng dần**: triệu dòng thô thành trăm nghìn dòng fact, rồi bảng tổng hợp, cuối cùng là vài chục KPI. Vì vậy thách thức của tầng Presentation không phải quét petabyte, mà là (1) phục vụ **nhiều người dùng đồng thời với độ trễ tương tác**, (2) giữ **nhất quán ngữ nghĩa** giữa số do hệ thống phân tán tính và số người dùng thấy, (3) **co giãn** theo tải truy cập rất không đều.

2.2. Data Lake và định dạng cột Apache Parquet

Data Lake lưu dữ liệu dạng tệp theo nguyên tắc *schema-on-read*; nhiều công cụ (Spark, pyarrow, DuckDB, Trino) đọc trực tiếp mà không cần máy chủ cơ sở dữ liệu. Các tầng silver, gold, analytics của pipeline Olist đều là thư mục Parquet.

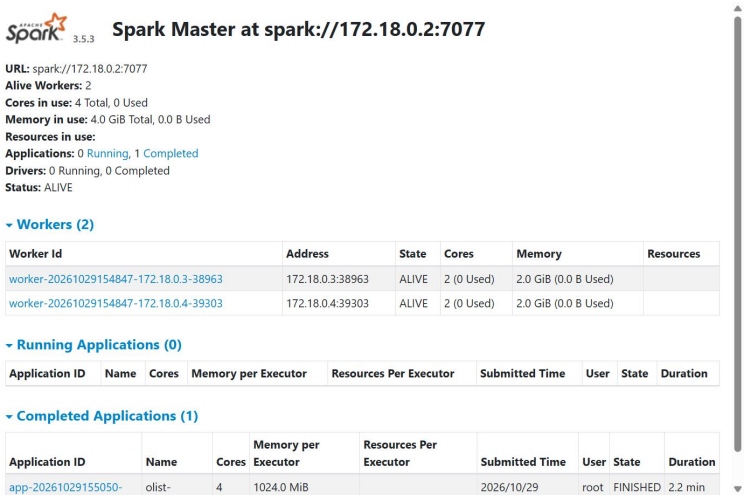
Apache Parquet [5] là định dạng hướng cột kế thừa Dremel [6]: tệp chia thành *row group*, trong đó dữ liệu từng cột nằm liền nhau (*column chunk*, chia thành *page*). Lợi ích [7]: chỉ đọc cột cần thiết; nén tốt nhờ mã hoá từ điển, run-length kết hợp Snappy (bảng `customer_segments` 94.983 dòng × 13 cột chỉ khoảng 3,9 MB); thống kê min/max cho phép bỏ qua khối dữ liệu (predicate pushdown); schema tự mô tả với kiểu logic `DECIMAL`, `DATE`, `TIMESTAMP` — nhờ đó loader của nhóm **tự sinh DDL PostgreSQL**. Tuy nhiên Parquet không phải hệ thống phục vụ: không có chỉ mục tra cứu điểm, không kiểm soát truy cập theo người dùng, không phục vụ nhiều client đồng thời — nên cần một tầng phục vụ riêng.

2.3. Xử lý phân tán bằng Apache Spark ở giai đoạn trước

Các bảng đầu vào được sinh bởi **Apache Spark** [3], [4] — công cụ xử lý phân tán trong bộ nhớ, kế thừa MapReduce [2] nhưng biểu diễn tính toán thành DAG các

phép biến đổi trên dữ liệu phân vùng. *Driver* lập kế hoạch, *executor* trên worker chạy *task* trên từng *partition*; `GROUP BY`, `JOIN`, window function sinh bước *shuffle*; trình tối ưu Catalyst chọn broadcast hoặc sort-merge join.

Nhóm Analytics chạy job trên cụm Docker 1 master + 2 worker (tổng 4 core), Spark 3.5.3, `spark.sql.shuffle.partitions = 16`. Job mất 119,71 giây (SQL 44,51 giây, K-Means 29,50 giây); bảng KPI nhỏ được `coalesce(1)` thành một tệp. Hình 2.1 là ảnh Spark UI do nhóm Analytics cung cấp, chứng minh job chạy phân tán. Lưu ý cho tầng Presentation: các window toàn cục (Pareto cộng dồn, NTILE của RFM) đã được Spark tính; nếu dashboard tính lại trên dữ liệu đã lọc sẽ ra kết quả khác, nên nhóm coi chúng là **thuộc tính đã chốt**.



Hình 2.1. *Spark Master UI (cổng 8080) của nhóm Analytics: 2 worker ALIVE, 4 core, ứng dụng olist-analytics FINISHED*

2.4. OLAP, bảng tổng hợp và tầng phục vụ

Hệ **OLTP** tối ưu cho giao dịch nhỏ theo dòng; hệ **OLAP** tối ưu cho truy vấn quét nhiều dòng và tổng hợp theo nhiều chiều [11]. Mô hình Kimball [10] tổ chức dữ liệu thành bảng **fact**, **dimension** và **bảng tổng hợp** tính trước theo các tổ hợp chiều thường dùng. Các bảng Analytics chính là bảng tổng hợp ở nhiều *grain*: tháng, danh mục, danh mục × tháng, bang, người bán, khách hàng. Trong kiến trúc Lambda [8], **tầng phục vụ** lập chỉ mục và phục vụ các *batch view* do tầng batch tính ra: ở đây Spark là batch layer, Neon là serving layer.

Khái niệm then chốt là **tính cộng dồn của độ đo** [10]: *cộng dồn được* (doanh thu, số đơn) có thể `SUM` theo mọi chiều; *bán cộng dồn* (số khách duy nhất) cộng theo tháng sẽ đếm trùng; *không cộng dồn* (tỷ lệ, trung bình, % tăng trưởng) không được lấy trung

bình của trung bình mà phải tính lại từ tử số và mẫu số. Tầng ngữ nghĩa của nhóm mã hoá các quy tắc này.

2.5. PostgreSQL và Neon Serverless Postgres

PostgreSQL là hệ quản trị quan hệ mã nguồn mở với SQL đầy đủ, giao dịch ACID, view, window function, chỉ mục và lệnh `COPY` nạp hàng loạt nhanh hơn nhiều lần so với `INSERT` từng dòng [14]; hầu hết công cụ BI đều kết nối được.

Neon là Postgres serverless **tách rời compute và storage** [12], tương tự Amazon Aurora [24]. *Compute* là tiến trình Postgres không trạng thái; *storage* gồm Safekeeper (nhân bản WAL bằng đồng thuận Paxos), Pageserver (dựng trang dữ liệu từ WAL, phục vụ compute) và object storage cho lưu trữ dài hạn. Các tính năng dùng trong đồ án [13]: **autoscaling** compute theo Compute Unit (≈ 1 vCPU + 4 GB RAM); **scale-to-zero** tạm dừng compute sau thời gian không hoạt động (mặc định 5 phút), truy vấn kế tiếp chịu độ trễ khởi động; **branching** copy-on-write để thử thay đổi trên dữ liệu thật; **connection pooling** qua hostname `-pooler` chạy PgBouncer chế độ transaction. Mô hình này khớp với **tải không đều** của dashboard: không ai xem thì compute ngủ, nhiều người xem thì compute tự tăng tài nguyên, dữ liệu luôn an toàn trong storage nhân bản.

2.6. Tầng ngữ nghĩa và Apache Superset

Tầng ngữ nghĩa định nghĩa *một lần* các khái niệm nghiệp vụ (chỉ số, chiều phân tích, tên hiển thị, cách tổng hợp) để mọi dashboard, báo cáo, API cho cùng một con số. Có thể đặt nó trong cơ sở dữ liệu (view), trong công cụ BI (dataset, metric) hoặc công cụ riêng (dbt, Cube). Nhóm đặt phần lõi ở **view SQL trên Neon** vì cả Superset lẫn giai đoạn Decision/Action đều đọc trực tiếp được.

Apache Superset [15] là nền tảng BI mã nguồn mở: web server Flask/Gunicorn với giao diện React và REST API; cơ sở dữ liệu metadata (SQLite/PostgreSQL); kết nối **SQLAlchemy** tới hầu hết hệ SQL; bộ nhớ đệm kết quả; tuỳ chọn Celery cho truy vấn bất đồng bộ và báo cáo định kỳ. Khái niệm chính: *Database*, *Dataset* (bảng/view hoặc SQL ảo), *Chart* (chủ yếu Apache ECharts), *Dashboard* (bố cục, tab), *Native filter* và *Cross-filter*. Superset đẩy toàn bộ phép tính xuống cơ sở dữ liệu, nên hiệu năng dashboard phụ thuộc trực tiếp vào tầng phục vụ và cache.

2.7. Nguyên tắc thiết kế dashboard

Thiết kế dựa trên nguyên tắc **"overview first, zoom and filter, then details on demand"** của Shneiderman [16] (thẻ KPI → biểu đồ → bộ lọc → bảng chi tiết) và khuyến nghị của Few [17]: xem được trong một màn hình, ưu tiên cột/đường hơn biểu đồ tròn và đồng hồ đo, màu nổi bật chỉ dành cho ngoại lệ. Biểu đồ được chọn theo câu hỏi: xu hướng → đường/cột theo tháng; tập trung 80/20 → Pareto; so sánh địa lý → bản đồ và scatter; xếp hạng → bảng có định dạng điều kiện. Mỗi biểu đồ ghi rõ nguồn (tên view), đơn vị và khoảng thời gian.

2.8. Các phương án công nghệ thay thế

Nhóm đã so sánh Superset với Power BI, Tableau, Grafana, Metabase và so sánh Neon với Postgres cục bộ, ClickHouse, đọc thẳng Parquet bằng DuckDB theo các tiêu chí giấy phép, triển khai, khả năng chia sẻ, vận hành và khả năng dựng dashboard bằng mã. Bảng so sánh đầy đủ ở Phụ lục B; lý do lựa chọn cuối cùng ở Mục 4.6.

3. Dữ liệu và yêu cầu

3.1. Nguồn dữ liệu và dòng dõi

Nguồn gốc là bộ **Brazilian E-Commerce Public Dataset by Olist** [1] trên Kaggle: 9 tệp CSV gồm đơn hàng (99.441 dòng), chi tiết đơn (112.650), thanh toán (103.886), đánh giá (99.224), khách hàng (99.441), người bán (3.095), sản phẩm (32.951), toạ độ mã bưu chính (1.000.163) và bảng dịch danh mục (71) — khoảng **1,55 triệu dòng, ~120 MB CSV**, đã ẩn danh hoá.

Bảng 3.1. Dòng dõi dữ liệu từ nguồn đến tầng phục vụ

Tầng	Nhóm phụ trách	Nội dung	Quy mô
Raw (CSV)	Sources → Ingestion	9 tệp CSV Olist	~1,55 triệu dòng, ~120 MB
Silver (Parquet)	Storage → Processing	Làm sạch, chuẩn hoá kiểu, khử trùng lặp	569.698 dòng, 40,37 MB
Gold (Parquet)	Storage → Processing	2 bảng fact + 5 bảng aggregate, đối chiếu 11/11	7 bảng, 33,3 MB, 61,72 s Spark
Analytics (Parquet)	Processing → Analytics	14 truy vấn Spark SQL + K-Means k = 4	16 bảng, 194.132 dòng, 119,71 s Spark
Serving (Neon)	**Analytics → Presentation**	Schema `analytics`, `gold`, `presentation`	23 bảng (505.592 dòng) + 22 view
Dashboard	**Analytics → Presentation**	Superset: KPI, biểu đồ, bộ lọc, gợi ý quyết định	Nhiều tab theo nhóm người dùng

Năm quy tắc nghiệp vụ từ tầng gold phải được giữ nguyên khi trình bày: (1) doanh thu chỉ tính trên đơn hợp lệ `is_valid_sale` (98.199/99.441 đơn); (2) đếm khách bằng `customer_unique_id` (99.441 `customer_id` nhưng chỉ 96.096 khách thật); (3) chuỗi thời gian chỉ dùng 2017-01 → 2018-08; (4) điểm review ở cấp đơn, không lấy trung bình theo dòng sản phẩm; (5) phân tích giao hàng loại đơn có cờ chất lượng `dq_*`.

3.2. Đặc trưng Big Data của bài toán

Đầu vào trực tiếp của nhóm ở mức trăm nghìn dòng, nhưng bài toán vẫn mang đặc trưng dữ liệu lớn và kiến trúc được thiết kế để giữ nguyên khi quy mô tăng:

- **Volume:** dữ liệu nguồn ~1,55 triệu dòng; nhóm Processing có sẵn kịch bản nhân bản ×5, ×10, ×20. Với một sàn thực tế (hàng chục triệu đơn/năm), bảng cấp khách hàng và người bán tăng tuyến tính lên hàng chục triệu dòng; tầng phục vụ phải chịu được mức này mà không đổi kiến trúc (Mục 8.3).
- **Variety:** 16 bảng với 10 grain khác nhau (tháng, danh mục, danh mục × tháng, bang, khoảng cách, khách hàng, cụm, cohort, người bán, nhóm thanh toán), nhiều

kiểu dữ liệu, hai hệ nhãn khách hàng song song (luật RFM và cụm K-Means), cộng thêm bảng fact gold phân vùng Hive.

- **Veracity:** cờ chất lượng `dq_*`, tháng không đầy đủ và các bẫy ngữ nghĩa — tổng 20 tháng của `kpi_monthly` *nhỏ hơn* số neo 13.494.400,74 BRL vì đã cắt tháng rỗng; `f_score` gần như nhiễu vì 97% khách chỉ mua một lần. Nhóm phải kiểm tra đối chiếu tự động để chứng minh số trên dashboard đúng.
- **Value:** kết quả phải chuyển thành hành động — 153 người bán cần theo dõi, ngưỡng 4 ngày trễ làm review sụt mạnh.
- **Nhu cầu phục vụ ở quy mô lớn:** nhiều người dùng, nhiều nơi, tải không đều → chọn dịch vụ serverless tự co giãn thay vì máy chủ cố định.

Velocity không phải đặc trưng chính: dữ liệu được làm mới theo lô mỗi khi Analytics chạy lại; loader idempotent cho phép tăng tần suất làm mới mà không đổi kiến trúc.

3.3. Input

Nguồn và cách thu thập: các bảng do nhóm Analytics bàn giao được sao chép nguyên vẹn vào `data/analytics/` của repo nhóm. **Định dạng:** Parquet nén Snappy, mỗi bảng một thư mục kiểu Spark (`part-00000-*.snappy.parquet`, kèm `_SUCCESS`). **Tốc độ phát sinh:** theo lô (lần chạy Analytics hiện tại: 29/10/2026). **Quy mô:** 16 bảng, 194.132 dòng, 137 cột, 8.033,0 KB nén; phân bố rất lệch — hai bảng cấp khách hàng (`rfm_scores`, `customer_segments`, mỗi bảng 94.983 dòng) và `seller_pareto` (3.053 dòng) chiếm hơn 99% số dòng, 13 bảng còn lại chỉ 2–715 dòng. Danh sách bảng, grain, khoá ở Mục 3.5; schema từng cột, quy mô chi tiết và ví dụ bản ghi ở Phụ lục C. Ví dụ rút gọn một dòng của `kpi_monthly`: `{purchase_year: 2017, purchase_month: 11, orders: 7421, revenue: 1003862.14, avg_order_value: 157.96, late_delivery_pct: 14.31, avg_review_score: 3.95, revenue_mom_pct: 52.06, revenue_rank: 1, is_black_friday_month: true}`.

Đầu vào bổ sung – tầng gold. Bảng Analytics có grain cố định nên không trả lời được câu hỏi lọc chéo như "AOV và % trễ của bang SP trong quý I/2018 với khách trả bằng boleto". Vì vậy nhóm nạp thêm 7 bảng gold (311.460 dòng, đầu vào của Analytics, đã đối chiếu 11/11). Hai bảng fact phân vùng Hive theo `purchase_year=YYYY/purchase_month=M/`; loader đọc bằng `pyarrow.dataset(..., partitioning="hive")` để khôi phục cột phân vùng. Gold chỉ dùng để **lọc và cộng lại**;

mọi KPI phân tích (Pareto, RFM, K-Means, watchlist) vẫn lấy nguyên từ Analytics, và bộ kiểm tra nhất quán (Mục 6.2) chứng minh tổng từ gold khớp tổng của `kpi_monthly`.

3.4. Output

Bảng 3.2. Các thành phần đầu ra của giai đoạn Presentation

Đầu ra	Định dạng / vị trí	Người dùng	Ví dụ
Bảng phục vụ	PostgreSQL 18 trên Neon: `analytics` (16 bảng + `load_log`), `gold` (7 bảng)	Superset, Decision/Action	`analytics.kpi_monthly` 20 dòng
Tầng ngữ nghĩa	22 view + bảng `dim_state` trong schema `presentation`	Superset, Decision/Action	`v_kpi_monthly` có `month_date` kiểu DATE
Dashboard	Apache Superset (HTTP cổng 8088), nhiều tab	Người dùng nghiệp vụ	Thẻ "Doanh thu", Pareto, bản đồ bang
Gợi ý quyết định	`v_decision_hints` (7 luật), `v_seller_watchlist` (ĐỎ/VÀNG)	Giai đoạn Decision/Action	153 người bán kèm lý do, hành động
Minh chứng	`evidence/`: log nạp, đối chiếu, độ trễ, ảnh	Giảng viên, nhóm	`load_log.csv`, `validation.json`

Ví dụ một dòng của `v_decision_hints`: *ưu tiên 2, mức CAO, chủ đề Giao hàng, tín hiệu "Đơn trễ kéo review xuống", giá trị 1,73, ngưỡng "chênh lệch > 1 sao", hành động "Cảnh báo seller khi đơn trễ 1–3 ngày".*

3.5. Data contract giữa Analytics và Presentation

Data contract là thoả thuận ràng buộc giữa bên sản xuất (Analytics) và bên tiêu thụ (Presentation) về **cấu trúc, ngữ nghĩa và chất lượng** dữ liệu bàn giao. Nhóm hình thức hoá `ANALYTICS_HANDOVER.md` thành bảng dưới đây (sinh tự động từ Parquet) và bộ kiểm tra tự động.

Bảng 3.3. Data contract Analytics → Presentation: danh sách bảng, grain và khoá

Bảng (analytics.*)	Chủ đề	1 dòng =	Khoá / cột định danh	Số dòng	Số cột	Dùng trên dashboard
`kpi_monthly`	1. Xu hướng	1 tháng (2017-01 → 2018-08)	purchase_year, purchase_month	20	17	KPI tổng quan, xu hướng
`category_pareto`	2. Danh mục	1 danh mục	category	74	9	Pareto danh mục
`category_seasonality`	2. Danh mục	1 danh mục × 1 tháng (2017)	category, purchase_month	715	5	Heatmap mùa vụ
`state_logistics`	3. Địa lý	1 bang của khách	customer_state	27	13	Bản đồ / bảng bang
`insight_distance`	3. Địa lý	1 khoảng cách (band km)	distance_band (band_order)	4	7	Logistics theo km
`rfm_scores`	4. Khách hàng	1 khách thật	customer_unique_id	94.983	12	Phân khúc RFM
`customer_segments`	4. Khách hàng	1 khách thật (+ cụm K-Means)	customer_unique_id	94.983	13	Phân khúc + cụm
`cluster_profile`	4. Khách hàng	1 cụm K-Means	cluster	4	8	Hồ sơ cụm
`repeat_profile`	4. Khách hàng	1 cohort (repeat / one_shot)	cohort	2	7	Thẻ KPI giữ chân

Bảng (analytics.*)	Chủ đề	1 dòng =	Khoá / cột định danh	Số dòng	Số cột	Dùng trên dashboard
repeat_categories	4. Khách hàng	1 danh mục của khách mua lại	category	70	4	Danh mục khách lặp
repeat_states	4. Khách hàng	1 bang của khách mua lại	customer_state	27	5	Bang khách lặp
seller_pareto	5. Người bán	1 người bán	seller_id	3.053	11	Pareto người bán
seller_watchlist	5. Người bán	1 người bán cần theo dõi	seller_id	153	10	Gợi ý quyết định
insight_late_vs_review	6. Giao hàng & review	1 nhóm (late / on_time)	delivery	2	5	So sánh trễ vs đúng hạn
delay_vs_review	6. Giao hàng & review	1 khoảng ngày trễ	delay_band (band_order)	5	6	Ngưỡng trễ → review
payment_mix	6. Giao hàng & review	1 PTTT × 1 nhóm kỳ trả góp	main_payment_type, installment_band	10	5	Cơ cấu thanh toán
Tổng				**194.132**	137	

Các điều khoản:

1. **Schema:** tên bảng, cột, kiểu như Phụ lục C. Thêm cột được chấp nhận (loader tự sinh DDL); đổi tên hoặc xoá cột là thay đổi phá vỡ, phải báo trước.
2. **Grain và khoá:** khoá ở cột "Khoá" là duy nhất — loader khai báo thành khoá chính, nên vi phạm grain bị phát hiện khi nạp.
3. **Ngữ nghĩa độ đo:** `revenue` = tổng giá sản phẩm chưa gồm ship; `gmv` = `revenue` + ship; `avg_review_score` cấp đơn; `late_delivery_pct` tính trên đơn đã giao; `delivery_delay_days` âm là giao sớm; tiền BRL 2 chữ số thập phân.
4. **Phạm vi thời gian:** 2017-01 → 2018-08; `category_seasonality` chỉ năm 2017.
5. **Số neo đối chiếu:** doanh thu gold 13.494.400,74 BRL; `kpi_monthly` 20 tháng; 94.983 khách; đỉnh 11/2017 với MoM +52,06%; review trễ/đúng hạn 2,56/4,29; khách mua lại 2.887 (3,04%).
6. **Làm mới:** Analytics chạy lại thì toàn bộ thư mục được thay thế; Presentation nạp lại toàn bộ và ghi `run_id` vào `load_log`.
7. **Ánh xạ kiểu:** `decimal(p,s)` → `NUMERIC(p,s)`, `int32` → `INTEGER`, `int64` → `BIGINT`, `double` → `DOUBLE PRECISION`, `bool` → `BOOLEAN`, `date32` → `DATE`, `timestamp` → `TIMESTAMP`, `string` → `TEXT`.

3.6. Yêu cầu chức năng và phi chức năng

Chức năng: (F1) nạp toàn bộ bảng vào tầng phục vụ đúng kiểu dữ liệu; (F2) ghi nhật ký mỗi lần nạp; (F3) định nghĩa KPI thống nhất, tổng hợp đúng; (F4) dashboard

có thẻ KPI, xu hướng, Pareto, bản đồ, bảng chi tiết và bộ lọc theo tháng, bang, danh mục, phân khúc; (F5) cung cấp gợi ý quyết định cho giai đoạn sau. **Phi chức năng:** (N1) đúng đắn 100% so với Parquet và số neo; (N2) truy vấn dashboard trung vị < 300 ms, p95 < 500 ms; (N3) tái lập bằng Docker Compose và script; (N4) TLS bắt buộc, bí mật chỉ trong `.env`; (N5) nằm trong gói miễn phí Neon và không đổi kiến trúc khi dữ liệu ×10. Mức đáp ứng được đánh giá ở Mục 6.7.

3.7. Định nghĩa KPI

Bảng 3.4. Định nghĩa KPI và quy tắc tổng hợp trên dashboard

KPI	Định nghĩa	Nguồn	Tổng hợp khi lọc nhiều phần tử
Doanh thu (BRL)	Tổng `price` đơn hợp lệ, chưa gồm ship	`kpi_monthly`, `v_orders`	`SUM(revenue)`
Số đơn	Số đơn hợp lệ	`kpi_monthly`, `v_orders`	`SUM(orders)` / `COUNT(*)`
AOV (BRL)	GMV / số đơn	`v_orders`	`SUM(order_value)` / `COUNT(*)` — không AVG của AOV tháng
Tăng trưởng MoM (%)	(t − (t−1)) / (t−1) × 100	`kpi_monthly`	Không tổng hợp; hiển thị theo tháng
% đơn giao trễ	Đơn trễ / đơn đã giao × 100	`v_orders.late_flag` (1/0/NULL)	`AVG(late_flag) × 100` — NULL (chưa giao) tự bị loại
Điểm review TB	Trung bình sao 1–5 cấp đơn	`v_orders.review_score`	`AVG(review_score)` cấp đơn
Tỷ trọng / % cộng dồn Pareto	Phần tử / tổng; cộng dồn giảm dần	`category_pareto`, `seller_pareto`	Tính sẵn trên toàn kỳ (window của Spark)
Phí ship / doanh thu (%)	`freight / revenue × 100`	`category_pareto`, `state_logistics`	`SUM(freight) / SUM(revenue)`
% khách mua lại	Khách `frequency ≥ 2` / tổng khách	`repeat_profile`	Trên toàn bộ 94.983 khách
Chênh lệch review	review đúng hạn − review trễ	`insight_late_vs_review`	4,29 − 2,56 = 1,73 sao
Người bán cần theo dõi	Top 10% doanh thu **và** (review < 4 hoặc giao > 15 ngày hoặc trễ > 10%)	`seller_watchlist`	`COUNT(*)` = 153

Một lựa chọn thiết kế đáng chú ý: `kpi_monthly` chỉ có **tỷ lệ** % trễ và review theo tháng, không có mẫu số; nếu thẻ KPI lấy trung bình các tỷ lệ tháng khi người dùng chọn nhiều tháng thì sai (trung bình của trung bình). Nhóm giải quyết bằng view cấp đơn `v_orders` từ gold: `late_flag` nhận 1, 0 hoặc NULL (chưa giao), nên `AVG(late_flag)` đúng định nghĩa trên mọi tổ hợp bộ lọc. Biểu đồ xu hướng theo tháng vẫn lấy nguyên số của `kpi_monthly`, và bộ kiểm tra nhất quán đảm bảo hai nguồn khớp nhau.

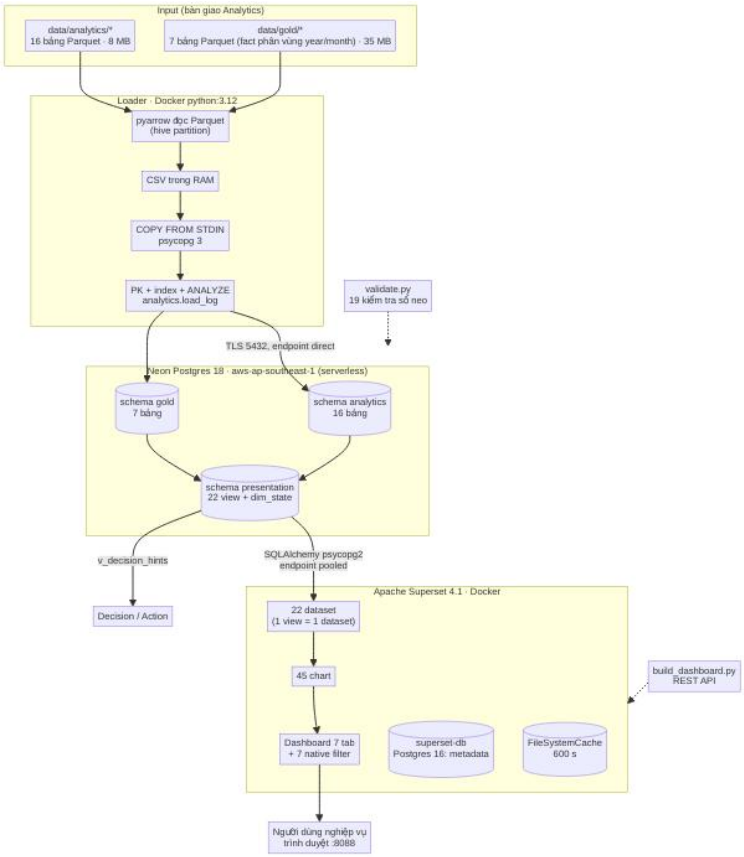
4. Thiết kế hệ thống

4.1. Kiến trúc tổng thể

Hệ thống gồm bốn khối, mỗi khối một trách nhiệm và giao tiếp qua định dạng/giao thức chuẩn (Hình 4.1):

1. **Data Lake (đầu vào):** 16 bảng Parquet Analytics (8 MB) và 7 bảng gold (35 MB, fact phân vùng năm/tháng).
2. **Loader (Python trong container `tools`):** pyarrow đọc Parquet → CSV trong bộ nhớ → `COPY FROM STDIN` (psycopg 3) qua endpoint *direct*, tạo khoá chính/chỉ mục, `ANALYZE`, ghi `load_log`, rồi áp dụng tầng ngữ nghĩa; `validate.py` kiểm 19 số neo.
3. **Tầng phục vụ (Neon Postgres 18, aws-ap-southeast-1):** schema `analytics` (16 bảng + `load_log`), `gold` (7 bảng), `presentation` (22 view + `dim_state`).
4. **Tầng trình bày (Apache Superset 4.1.2 trong Docker, 4 worker Gunicorn):** kết nối Neon qua SQLAlchemy + psycopg2 (endpoint *pooled*); 22 dataset (1 view = 1 dataset), 45 chart, dashboard 7 tab, 7 native filter; metadata lưu trong container Postgres 16 (`superset-db`); cache kết quả 600 s. Dashboard được dựng bằng `build_dashboard.py` qua REST API.

Người dùng nghiệp vụ mở dashboard qua trình duyệt (cổng 8088); giai đoạn Decision/Action đọc trực tiếp view `v_decision_hints` trên Neon.



Hình 4.1. *Sơ đồ data-flow của hệ thống: Parquet → Loader (COPY, TLS 5432 direct) → Neon (gold, analytics → presentation) → Superset (SQLAlchemy, endpoint pooled) → người dùng; nhánh v_decision_hints → Decision/Action*

4.2. Luồng dữ liệu, định dạng và giao thức

Bảng 4.1. Các bước của luồng dữ liệu

Bước	Từ → Đến	Định dạng / giao thức	Nơi xử lý	Mô tả
1	Spark → Data Lake	Parquet + Snappy	Cụm Spark (nhóm trước)	16 bảng Analytics; gold phân vùng Hive năm/tháng
2	Data Lake → Loader	Arrow (cột) → CSV trong RAM	Container `tools`	`pyarrow.dataset` đọc cả bảng phân vùng; ép timestamp ns → µs
3	Loader → Neon	Postgres v3, `COPY … FROM STDIN (FORMAT csv)`, TLS, cổng 5432, endpoint direct	Neon compute	DROP/CREATE/COPY trong một giao dịch; PK, index, `ANALYZE`, `load_log`
4	`analytics`, `gold` → `presentation`	SQL view	Neon compute	Tạo lại schema `presentation` trong một giao dịch
5	Neon → Superset	SQLAlchemy + psycopg2, TLS, endpoint pooled (PgBouncer)	Superset	Mỗi chart một truy vấn đẩy xuống Neon; cache 600 s
6	Superset → người dùng	HTTP 8088, JSON + React/ECharts	Trình duyệt	7 tab, native filter, cross-filter
7	Neon → Decision/Action	SQL (`v_decision_hints`, `v_seller_watchlist`), CSV, Chart Data API	Giai đoạn sau	Luật, ngưỡng, danh sách người bán cần xử lý

Nguyên tắc chung là **"tính một lần, đọc nhiều lần"**: phép tính nặng (join fact, window toàn cục, K-Means) đã làm ở Spark; tầng phục vụ chỉ lọc và tổng hợp nhẹ, nên độ trễ chủ yếu do mạng và kết nối (Chương 6).

4.3. Thiết kế tầng lưu trữ trên Neon

Bảng 4.2. Cấu hình tầng phục vụ Neon (theo `evidence/neon_project.json`)

Thuộc tính	Giá trị	Lý do
Project	`bigdata-olist-presentation` (muddy-wave-64763270), gói free_v3	Một project riêng cho giai đoạn Presentation
Vùng / phiên bản	`aws-ap-southeast-1` (Singapore), PostgreSQL 18.6	Vùng gần Việt Nam nhất
Compute	0,25 – 0,25 CU, scale-to-zero khi không hoạt động	Giới hạn gói miễn phí; không tốn compute khi không ai xem
Dung lượng	146 MB (25 bảng + 22 view), giới hạn 1 GiB/nhánh	Đủ cho dữ liệu đồ án và thí nghiệm ×10
Database / schema	`olist_analytics`: `analytics`, `gold`, `presentation`	Phân tầng vật lý – ngữ nghĩa
Kết nối	Loader: endpoint *direct*; Superset và đo độ trễ: endpoint *pooled*	`COPY`/DDL cần phiên ổn định; BI cần nhiều kết nối ngắn

Tách ba schema thể hiện nguyên tắc **phân tầng**: `analytics` và `gold` phản ánh cấu trúc do nhóm trước quyết định, còn `presentation` là giao diện ổn định cho người dùng và nhóm sau; khi nhóm trước đổi tên cột, chỉ cần sửa view.

DDL tự sinh và ánh xạ kiểu. Loader không chứa `CREATE TABLE` viết tay: DDL được sinh từ schema Arrow; kiểu chưa ánh xạ gây lỗi ngay. **Tiền tệ giữ `NUMERIC(p,s)`** để tổng 13.494.400,74 BRL khớp đến từng xu; **timestamp nano giây ép về micro giây**; **chuỗi rỗng và NULL được phân biệt** bằng cách đặt mọi giá trị hợp lệ trong dấu nháy khi ghi CSV. Mỗi bảng có `COMMENT ON TABLE` ghi grain và nguồn.

Nạp idempotent, nhật ký, chỉ mục. Mỗi lần chạy có một `run_id`; với từng bảng, `DROP … CASCADE`, `CREATE`, `COPY` (khối 1 MiB) và `COMMENT` nằm trong **một giao dịch**, nên người dùng luôn thấy bản cũ hoặc bản mới hoàn chỉnh. Sau đó loader tạo khoá chính theo grain của data contract (vi phạm thì lùi về chỉ mục thường), chỉ mục phụ trên cột bộ lọc của bảng lớn (`rfm_segment`, `customer_state`, `cluster`, `seller_state`, `purchase_date`, `category`, `seller_id`, `main_payment_type`), `ANALYZE`, rồi so `count(*)` với Parquet; lệch thì kết thúc với mã lỗi. `analytics.load_log` lưu mỗi bảng mỗi lần nạp một dòng; view `v_load_log` hiển thị độ tươi và nguồn gốc dữ liệu trên dashboard.

4.4. Thiết kế tầng ngữ nghĩa

Tầng ngữ nghĩa (`sql/10_presentation_views.sql`) bắt đầu bằng `DROP SCHEMA IF EXISTS presentation CASCADE` rồi tạo lại toàn bộ, nên luôn nhất quán với dữ liệu vừa nạp. Bốn quy tắc: (1) **không tính lại** những gì Spark đã tính (Pareto, NTILE, K-Means), chỉ gắn nhãn và thêm cột dẫn xuất nhẹ; (2) **thêm cột phục vụ trình bày** — `month_date` kiểu DATE cho bộ lọc thời gian, `dim_state` với tên bang, vùng và `iso_code = 'BR-' || customer_state` cho Country Map, nhãn tiếng Việt, tên nghiệp vụ cho 4 cụm K-Means; (3) **giữ quy tắc bàn giao** — khung 2017-01 → 2018-08, đơn hợp lệ, đếm `customer_unique_id`, review chỉ ở cấp đơn; (4) **mã hoá gợi ý quyết định** thành dữ liệu (số vi phạm, lý do, mức ĐỎ/VÀNG, hành động).

Các view được nhóm theo chủ đề: **xu hướng** (`v_kpi_monthly`, `v_kpi_overview`, `v_orders`), **danh mục** (`v_category_pareto`, `v_category_seasonality`, `v_category_monthly`, `v_sales_items`), **địa lý** (`v_state_logistics`, `v_insight_distance`), **khách hàng** (`v_rfm_customers`, `v_rfm_segments`, `v_cluster_profile`, `v_repeat_*`), **người bán** (`v_seller_pareto`, `v_seller_watchlist`), **giao hàng – thanh toán** (`v_late_vs_review`, `v_delay_vs_review`, `v_payment_mix`), **quyết định** (`v_decision_hints`) và **nguồn gốc dữ liệu** (`v_load_log`). Danh mục đầy đủ ở Phụ lục D.

Hai lớp dữ liệu cho bộ lọc. Bảng Analytics có grain cố định (`category_pareto` không có tháng, `kpi_monthly` không có bang) nên không lọc chéo được. Lớp **cấp đơn/sản phẩm** (`v_orders`, `v_sales_items`, `v_category_monthly` từ gold) phục vụ thẻ KPI và xu hướng cần lọc tự do theo tháng × bang × danh mục × PTTT; lớp **insight** (view trên `analytics.*`) phục vụ kết quả phân tích toàn kỳ như Pareto, RFM, watchlist. Ví dụ: `AVG(late_flag)` trên `v_orders` cho tỷ lệ trễ đúng định nghĩa trên mọi tổ hợp bộ lọc, thay vì trung bình các tỷ lệ tháng.

4.5. Thiết kế dashboard và giao diện với Decision/Action

Dashboard "Olist E-commerce · Analytics → Presentation" gồm bảy tab theo sáu chủ đề của Analytics cộng một tab quyết định; mỗi tab theo mẫu "tổng quan → chi tiết" (thẻ KPI → biểu đồ → bảng).

Bảng 4.3. Thiết kế dashboard theo tab và nhóm người dùng

Tab	Người dùng chính	Câu hỏi nghiệp vụ	Thành phần chính

Tab	Người dùng chính	Câu hỏi nghiệp vụ	Thành phần chính
1 · Tổng quan & xu hướng	Ban điều hành	Doanh thu tăng thế nào? Vận hành có theo kịp?	6 thẻ KPI, doanh thu & đơn theo tháng, % trễ vs review, MoM, bảng KPI tô màu
2 · Danh mục & mùa vụ	Quản lý ngành hàng	Ngành nào tạo 80% doanh thu? Mùa vụ?	Pareto top 25, heatmap mùa vụ 2017, ship/doanh thu, xu hướng 6 ngành lớn
3 · Địa lý & logistics	Vận hành – logistics	Bang nào giao chậm, trễ nhiều, review thấp?	Country Map Brazil, bubble ngày giao vs review, % trễ theo bang, band khoảng cách
4 · Khách hàng & RFM	CRM – marketing	Bao nhiêu khách quay lại? Chăm sóc phân khúc nào?	4 thẻ, donut phân khúc RFM, cụm K-Means, bảng hành động CRM
5 · Seller	Quản lý người bán	Ai tạo 80% doanh thu? Ai lớn nhưng vận hành kém?	Pareto nhóm A/B/C, bubble top 300, bảng watchlist cảnh báo
6 · Giao trễ, review & thanh toán	Vận hành, CSKH, tài chính	Trễ bao nhiêu ngày thì review sụt?	Review trễ vs đúng hạn, ngưỡng ngày trễ vs % 1 sao, payment mix
7 · Gợi ý quyết định & nguồn dữ liệu	Ban điều hành, nhóm Decision	Cần làm gì trước? Dữ liệu nạp lúc nào?	Bảng 7 luật, nhật ký nạp

Bảy native filter — Khoảng thời gian (`month_date`), Bang khách hàng, Vùng, Danh mục, Phân khúc RFM, Phương thức thanh toán, Bang seller — chỉ gắn vào chart có cột tương ứng (phạm vi do script tính tự động). Màu nổi bật chỉ dành cho ngoại lệ (tháng Black Friday, ô review < 4, trễ > 10%); không dùng biểu đồ tròn cho 74 danh mục hay 27 bang.

Giao diện với Decision/Action. View `v_decision_hints` có mỗi dòng là một luật: ưu tiên, mức độ, chủ đề, tín hiệu, giá trị hiện tại, ngưỡng, hành động và view nguồn. Giá trị được tính lại mỗi lần truy vấn nên luật tự cập nhật khi dữ liệu được nạp lại.

Bảng 4.4. Bảy luật gợi ý quyết định trong v_decision_hints

#	Mức	Chủ đề	Tín hiệu / ngưỡng	Hành động khuyến nghị
1	CAO	Người bán	Top 10% doanh thu vi phạm ≥ 2 tiêu chí (review < 4, giao > 15 ngày, trễ > 10%): 75/153	Cảnh cáo, giảm hiển thị, kế hoạch SLA 30 ngày
2	CAO	Giao hàng	Review đúng hạn 4,29 vs trễ 2,56 (chênh > 1 sao)	Cảnh báo người bán khi đơn trễ 1–3 ngày
3	CAO	Vận hành	Tháng trễ > 15%: 03/2018 (21,36%), 02/2018 (15,99%)	Tăng năng lực vận chuyển trước cao điểm
4	TRUNG BÌNH	Địa lý	9 bang trễ > 10% và review < 4	Kho/người bán vùng; báo ngày giao thực tế
5	TRUNG BÌNH	Danh mục	Ngành > 100k BRL có phí ship > 20% doanh thu	Không free-ship đại trà
6	TRUNG BÌNH	Khách hàng	1.961 khách At Risk	Win-back có mục tiêu
7	THẤP	Người bán	17,65% người bán tạo 80% doanh thu	Account manager cho nhóm A

4.6. Lựa chọn công nghệ và lý do

Bảng 4.5. Công nghệ sử dụng, vai trò và lý do lựa chọn

Công nghệ	Vai trò trong kiến trúc	Lý do chọn
Parquet + pyarrow	Định dạng nhận từ Analytics; đọc cả thư mục phân vùng Hive	Giữ kiểu Spark (decimal, date); không cần Spark để đọc bảng đã cô đọng
Python loader + psycopg 3 (`COPY`) trong `python:3.12-slim`	Tạo bảng đúng kiểu, nạp hàng loạt, PK/index, `load_log`	`COPY` nhanh hơn INSERT nhiều lần; máy Windows không cần cài Python; chạy lại được
Neon Postgres 18 (serverless)	Serving layer: `analytics`, `gold`, `presentation`	SQL chuẩn, tách compute/storage, scale-to-zero, chia sẻ qua Internet + TLS, không phải tự vận hành
View SQL	Semantic layer, luật cảnh báo	KPI định nghĩa một nơi, có version control, dùng chung cho dashboard và Decision
Apache Superset 4.1.2 (Docker, 4 worker)	Dashboard, chart, native filter, cache	Mã nguồn mở, REST API để dựng dashboard bằng mã, Country Map Brazil
Postgres 16 (`superset-db`)	Metadata của Superset	SQLite khoá ghi toàn cục khiến chart tải song song phải xếp hàng
Playwright (Chromium headless)	Chụp ảnh, đo thời gian tải dashboard và từng chart	Tự động, nhất quán, chạy lại được

Vì sao Superset? So với Power BI/Tableau (thương mại, tệp nhị phân khó quản lý phiên bản), Grafana (mạnh về chuỗi thời gian giám sát, yếu với Pareto/xếp hạng) và Metabase (ít loại biểu đồ), Superset chạy bằng Docker, kết nối thẳng Postgres và cho phép **dựng dashboard bằng mã** — then chốt cho yêu cầu tái lập. **Vì sao Neon?** Postgres cục bộ không chia sẻ được cho nhóm, giảng viên và giai đoạn sau; ClickHouse chỉ phát huy ở hàng tỷ dòng chi tiết; DuckDB không có máy chủ phục vụ đồng thời. Neon cho Postgres chuẩn, được quản lý, khớp tải không đều của dashboard. Cấu hình chi tiết (Compose, `superset_config.py`) ở Phụ lục F.

5. Cài đặt và triển khai

5.1. Môi trường và cấu trúc mã nguồn

Bảng 5.1. Môi trường cài đặt và triển khai

Thành phần	Phiên bản / cấu hình
Máy chạy	Windows 11, PowerShell, Docker Desktop (Docker 29, Compose v2); mạng Internet tại Việt Nam → Neon Singapore (RTT ~105 ms)
Container `tools`	`python:3.12-slim` + pyarrow 17, pandas 2.2, psycopg 3.2, SQLAlchemy 2.0, requests
Superset	`apache/superset:4.1.2` + `psycopg2-binary`, 4 worker Gunicorn, cache filesystem 600 s
Metadata Superset	`postgres:16-alpine` (container `superset-db`, volume riêng)
Ảnh chụp / đo tải	Playwright + Chromium headless, viewport 1600×1900
Tầng phục vụ	Neon PostgreSQL 18.6, `aws-ap-southeast-1`, 0,25 – 0,25 CU, gói free_v3
Đầu vào	16 bảng Analytics (194.132 dòng) + 7 bảng gold (311.460 dòng)

Repo được tổ chức theo thành phần kiến trúc: `data/` (Parquet đầu vào), `loader/` (nạp, áp dụng SQL, kiểm chứng), `sql/` (tầng ngữ nghĩa, 19 kiểm tra số neo), `superset/` (Dockerfile, cấu hình, `build_dashboard.py`, `import_dashboard.py`, bản export ZIP), `scripts/` (đo độ trễ, thí nghiệm scale, chụp ảnh, vẽ sơ đồ, kiểm kê Neon), `evidence/`, `docs/`, `report/`. Cây thư mục đầy đủ ở Phụ lục F. Mọi script chạy trong container nên máy Windows không cần cài Python, và mọi thành viên dùng cùng phiên bản thư viện.

5.2. Khởi tạo Neon và nạp dữ liệu

Nhóm tạo project `bigdata-olist-presentation` (PostgreSQL 18, AWS Singapore) và database `olist_analytics`; chuỗi kết nối pooled được ghi vào `NEON_DATABASE_URL` trong `.env` (không commit). Loader `loader/load_to_neon.py` khai báo tường minh 23 bảng cùng khoá chính, chỉ mục và grain trong từ điển `TABLES`. Một lần chạy: (1) tự bỏ hậu tố `-pooler` để dùng endpoint direct, ghi phiên bản Postgres và chuỗi kết nối đã che mật khẩu vào `evidence/load_run.log`; (2) tạo schema và `load_log`; (3) với mỗi bảng, đọc bằng `pyarrow.dataset` (nhận phân vùng Hive), sinh DDL, chuyển CSV trong bộ nhớ; (4) nạp trong một giao dịch; (5) tạo PK/chỉ mục, `ANALYZE`, đếm dòng, ghi `load_log`; (6) áp dụng `sql/[0-8]*.sql`; (7) xuất `evidence/load_log.csv`. Lõi phần nạp:

```
table, nfiles, nbytes = read_table(src)          # pyarrow.dataset,
partitioning="hive"
payload = to_csv_bytes(table)                     # pyarrow.csv,
quoting_style="all_valid"
```

```
with conn.transaction():
    with conn.cursor() as cur:
        cur.execute(f"DROP TABLE IF EXISTS {fq} CASCADE")
        cur.execute(f"CREATE TABLE {fq} (\n    {cols_ddl}\n)")
        with cur.copy(f"COPY {fq} ({collist}) FROM STDIN WITH (FORMAT csv)") as cp:
            view = memoryview(payload)
            for i in range(0, len(view), 1 << 20):   # khối 1 MiB
                cp.write(view[i : i + (1 << 20)])
```

Sau khi nạp, `loader/validate.py` đọc lại Parquet để so số dòng với Postgres và chạy 19 truy vấn trong `sql/90_anchor_checks.sql` (mỗi truy vấn kèm chú thích `-- check: tên | expected | tol`), xuất `evidence/validation.md/.json`. Khi chỉ sửa view, chạy `loader/apply_sql.py` (tạo lại 22 view + `dim_state` trong dưới một giây).

5.3. Triển khai Apache Superset

Superset được đóng gói từ image `apache/superset:4.1.2` cài thêm `psycopg2-binary`; `docker-init.sh` chạy `superset db upgrade`, tạo tài khoản quản trị từ biến môi trường, `superset init` rồi khởi động web server với 4 worker — mọi bước idempotent. Metadata nằm trong container Postgres 16 `superset-db` thay vì SQLite. `superset/build_dashboard.py` dựng toàn bộ dashboard qua **REST API** (dashboard-as-code): lấy token, tạo *Database* trỏ tới Neon, đăng ký 22 view thành dataset, tạo 45 chart, dashboard 7 tab và 7 native filter kèm phạm vi; chạy lại sẽ cập nhật thay vì tạo trùng. Bản export `superset/export/dashboard_olist_presentation.zip` cùng `import_dashboard.py` là phương án dự phòng. Nhờ vậy dashboard dựng lại giống hệt trên máy khác chỉ bằng một lệnh.

5.4. Xử lý lỗi và các vấn đề kỹ thuật

Bảng 5.2. Các vấn đề kỹ thuật đã gặp và cách xử lý

Vấn đề	Nguyên nhân	Cách xử lý
Máy Windows không có Python	Môi trường không đồng nhất	Container `tools` chạy mọi script
`COPY`/DDL qua endpoint pooled có thể lỗi	PgBouncer chế độ transaction không giữ phiên	Loader tự đổi sang endpoint direct
Superset không kết nối được Postgres	Image gốc không có driver	Cài `psycopg2-binary` trong Dockerfile
Script khởi động lỗi `\r`	Tệp soạn trên Windows dùng CRLF	`sed -i 's/\r$//'` trong Dockerfile
Chart tải song song phải xếp hàng	Metadata SQLite khoá ghi toàn cục	Chuyển metadata sang Postgres 16, 4 worker Gunicorn
Lần mở dashboard đầu chậm (~0,7 s/chart)	Superset dùng NullPool, mỗi chart mở kết nối TLS mới	Cache kết quả 600 s; endpoint pooled; đề xuất đặt Superset cùng vùng
Tổng tiền lệch vài xu / timestamp không hợp lệ	Số thực cho tiền; Arrow lưu nano giây	`NUMERIC(p,s)`; ép timestamp về micro giây

Vấn đề	Nguyên nhân	Cách xử lý
Chuỗi rỗng bị hiểu thành NULL; mất cột năm/tháng	CSV không phân biệt; cột phân vùng nằm trong tên thư mục	`quoting_style="all_valid"`; `partitioning="hive"`
Bộ lọc thời gian / bản đồ không hoạt động	Thiếu cột DATE; Country Map cần mã ISO	`month_date`; `dim_state.iso_code = 'BR-xx'`
Bộ lọc vừa dựng bị ghi đè	Tab trình duyệt đang mở lưu lại trạng thái dashboard cũ	Tải lại trang sau khi chạy `build_dashboard.py`

Các lệnh chạy end-to-end (build, nạp, kiểm chứng, dựng dashboard, tái tạo minh chứng) ở Phụ lục A.

6. Thực nghiệm và đánh giá

6.1. Kịch bản, môi trường và tiêu chí

Thực nghiệm trả lời RQ2 (tính đúng đắn), RQ3 (hiệu năng) và cung cấp số liệu cho RQ5 (mở rộng).

Bảng 6.1. Các kịch bản thực nghiệm

Mã	Kịch bản	Cách thực hiện	Đại lượng đo
E1	Đối chiếu Parquet ↔ Neon	`validate.py`: đếm dòng 23 bảng; 19 truy vấn số neo/nhất quán	Số kiểm tra đạt
E2	Hiệu năng nạp	`load_to_neon.py` ghi thời gian từng pha mỗi bảng	Giây, dòng/s, MB/s
E3	Độ trễ truy vấn	`measure_latency.py`: 21 truy vấn của dashboard qua endpoint pooled, 5 lần sau 1 lần làm nóng	Trung vị, p95 client; thời gian server
E4	Thời gian tải dashboard	Playwright mở 7 tab sau khi xoá cache (cold) và mở lại (cache); replay 44 chart qua REST API	Giây/tab, ms/chart
E5	Bộ lọc native	6 kịch bản lọc trên 6 tab, chụp ảnh và đo thời gian	Đạt/không, giây
E6	Mở rộng ×1 → ×10	`scale_experiment.py`: nhân bản `fact_order_items` trên Neon, `EXPLAIN ANALYZE`	ms server theo hệ số

Môi trường. Client là các container Docker trên máy Windows 11 tại Việt Nam; Neon ở Singapore với 0,25 – 0,25 CU (gói miễn phí). Mọi số đo phía client bao gồm độ trễ mạng (RTT `SELECT 1` ≈ 105 ms) — đúng với kịch bản sử dụng thực tế của nhóm.

Tiêu chí đạt: đúng đắn 100%; nạp 16 bảng Analytics < 120 s; truy vấn dashboard có trung vị < 300 ms và p95 < 500 ms; một tab dashboard < 5 s khi cold và < 2 s khi có cache; 100% kịch bản bộ lọc phản hồi đúng; truy vấn view tổng hợp không tăng theo kích thước dữ liệu nguồn.

6.2. E1 – Đối chiếu dữ liệu

`validate.py` cho kết quả **23/23 bảng khớp số dòng** và **19/19 kiểm tra số neo/nhất quán đạt** (một dòng INFO chỉ hiển thị tổng khung 20 tháng). Ba nhóm kiểm tra: (1) **số neo bàn giao** — tổng doanh thu gold 13.494.400,74 BRL (tính cả từ `agg_sales_monthly` lẫn `fact_order_items`), tháng đỉnh 2017-11 với MoM 52,06%, review 2,56/4,29, 2.887 khách mua lại (3,04%), 153 người bán watchlist, 539 người bán tạo 80% doanh thu; (2) **nhất quán chéo** — tổng doanh thu và số đơn của `v_orders`, tổng doanh thu của `v_sales_items` (từ gold) bằng đúng tổng của `kpi_monthly` (từ Analytics), chênh lệch 0,00; (3) **toàn vẹn tầng ngữ nghĩa** — tổng khách theo phân khúc bằng 94.983, `v_decision_hints` đủ 7 luật. Nhóm (2) chứng minh việc bổ sung lớp gold để lọc chéo không tạo ra "hai sự thật". Tổng 20 tháng của

`kpi_monthly` (13.449.529,68 BRL) nhỏ hơn số neo vì đã loại các tháng 2016 và 09/2018 — thẻ KPI vì vậy ghi rõ khung thời gian. Bảng chi tiết ở Phụ lục E.

6.3. E2 – Hiệu năng nạp dữ liệu

Lần chạy `20261112-145508-40efaf` nạp 23 bảng, 505.592 dòng (42,88 MB Parquet) trong 351,34 giây (pha COPY 330,80 giây), thông lượng đầu-cuối 1.439 dòng/s. Riêng 16 bảng Analytics mất 72,66 giây; 7 bảng gold mất 278,68 giây (~79%). Ba quan sát: (1) **chi phí cố định mỗi bảng** — 15 bảng dưới 1.000 dòng mất trung bình 1,73 giây mỗi bảng do nhiều vòng khứ hồi tới Singapore (DDL, COPY, commit, PK, ANALYZE, đếm), nên cần gộp giao dịch hoặc nạp song song; (2) **bảng lớn bị giới hạn bởi băng thông tải lên** — CSV truyền đi lớn hơn Parquet 2,5–3 lần (107,36 MB so với 42,88 MB) với tốc độ ~0,32 MB/s, tức vài Mbit/s của đường truyền thông thường, không phải giới hạn của Postgres; (3) **lớp gold chiếm phần lớn thời gian** — chi phí của khả năng lọc chéo. Hướng tối ưu hợp lý nhất là chạy loader cùng vùng với Neon. Bảng từng bảng ở Phụ lục E.

6.4. E3 – Độ trễ truy vấn

Bảng 6.2. Tóm tắt độ trễ truy vấn (`query_latency.md`, `superset_chart_latency.csv`)

Đại lượng	Giá trị
RTT mạng `SELECT 1` (Việt Nam → Singapore)	105 ms
Thiết lập kết nối TLS mới tới Neon	1.134 ms
21 truy vấn dashboard – trung vị phía client / p95 lớn nhất	173,4 ms / 444,3 ms
21 truy vấn dashboard – trung vị thực thi trên server	1,25 ms
Pareto danh mục: đọc view tổng hợp / tính lại từ `fact_order_items` (server)	0,24 ms / 66,43 ms
Chart qua Superset API (44/44 chart): bỏ cache / từ cache (trung vị)	741 ms / 55 ms

Ba kết luận: (1) **mạng chiếm gần như toàn bộ độ trễ** — Postgres thực thi các view insight dưới 3 ms, phần còn lại là đường truyền; các view cấp đơn/khách hàng (`v_orders`, `v_rfm_segments`, `v_decision_hints`) tốn 50–170 ms server nhưng vẫn dưới ngưỡng; (2) **bảng tổng hợp sẵn của tầng Big Data trả giá trị rõ rệt** — đọc Pareto đã tính ở Spark nhanh hơn tính lại từ fact hơn 250 lần; (3) **chi phí kết nối lớn hơn chi phí truy vấn** — Superset dùng NullPool nên mỗi chart mở kết nối TLS mới, khiến một chart không cache mất ~741 ms; cache đưa con số này về ~55 ms (nhanh ~14 lần). Bảng 22 truy vấn ở Phụ lục E.

6.5. E4, E5 – Thời gian tải dashboard và bộ lọc

Bảng 6.3. Thời gian hiển thị từng tab dashboard: cold và có cache (`evidence/dashboard_load_time.csv`)

Tab	Thành phần	Cold – không cache (s)	Có cache (s)
1 · tong quan xu huong	11	6,39	0,91
2 · danh muc mua vu	4	2,24	0,94
3 · dia ly logistics	5	12,66	0,95
4 · khach hang rfm	10	9,93	1,18
5 · seller	7	5,16	2,08
6 · giao tre review thanh toan	6	5,24	1,06
7 · goi y quyet dinh nguon du lieu	4	2,51	1,04
Trung vị / tổng 7 tab	47	**5,24 / 44,13**	**1,04 / 8,16**

Khung dashboard (HTML + JS) tải trong ~2,9 giây; không chart nào lỗi ở cả hai lượt. Khi cold, trung vị một tab là 5,24 giây; tab Địa lý chậm nhất (12,66 s) vì các chart xếp hàng chờ kết nối mới chứ không vì truy vấn nặng. Khi có cache, trung vị còn 1,04 giây, đủ cho người dùng nghiệp vụ. Compute Neon ở chế độ scale-to-zero; lượt cold ở trên đã bao gồm chi phí mở kết nối mới, còn thời gian đánh thức riêng compute không được đo tách biệt — theo tài liệu Neon [13] là cỡ vài trăm mili giây, nhỏ so với chi phí kết nối đo được.

Bảng 6.4. Kiểm thử bộ lọc native (ảnh `evidence/filter_*.png`, thời gian từ `dashboard_load_time.csv`)

Kịch bản bộ lọc native	Thành phần	Thời gian áp dụng (s)	Lỗi	Kết quả
Tab 1: khoảng thời gian 2018 + bang SP	11	2,50	0	Đạt
Tab 2: 3 danh mục (bed_bath, health_beauty, watches)	11	0,88	0	Đạt
Tab 3: vùng Nordeste	11	1,84	0	Đạt
Tab 4: phân khúc khách mua lại + bang RJ	11	1,55	0	Đạt
Tab 5: bang seller SP	11	0,88	0	Đạt
Tab 6: phương thức thanh toán boleto	11	1,52	0	Đạt

Cả 6/6 kịch bản bộ lọc phản hồi đúng: các chart trong phạm vi được tính lại theo bộ lọc (ví dụ lọc 2018 + bang SP đưa thẻ doanh thu từ 13.449.530 xuống 2.960.326 BRL, số đơn từ 97.905 xuống 23.598, Hình 7.3), chart ngoài phạm vi giữ nguyên, không lỗi; thời gian áp dụng 0,88–2,50 giây.

6.6. E6 – Thí nghiệm mở rộng

Bảng 6.5. Thí nghiệm mở rộng trên Neon 0,25 CU: nhân bản `gold.fact_order_items` ×N (`evidence/scale_experiment.csv`)

Hệ số	Số dòng	MB	Sinh dữ liệu (s)	Tổng hợp từ dữ liệu thô (ms)	Lọc có index (ms)	View tổng hợp sẵn (ms)
×1	112.101	12,7	0,18	47,0	2,5	1,1

Hệ số	Số dòng	MB	Sinh dữ liệu (s)	Tổng hợp từ dữ liệu thô (ms)	Lọc có index (ms)	View tổng hợp sẵn (ms)
×2	224.202	24,2	0,25	93,5	4,8	1,1
×5	560.505	59,2	0,57	272,9	11,3	1,1
×10	1.121.010	117,7	1,16	506,6	18,5	1,1

Khi dữ liệu tăng từ ×1 lên ×10 (1.121.010 dòng), truy vấn tổng hợp tháng × danh mục trên dữ liệu thô tăng gần tuyến tính từ 47,0 lên 506,6 ms; truy vấn có lọc dùng chỉ mục tăng chậm hơn nhiều (tới 18,5 ms); còn đọc view đã tổng hợp ở Spark giữ **1,1 ms bất kể kích thước nguồn**. Đây là bằng chứng định lượng cho thiết kế: Presentation đọc output Analytics thay vì tính lại từ fact, nên độ trễ dashboard không tăng theo dữ liệu nguồn. Thí nghiệm dừng ở ×10 do giới hạn 1 GiB/nhánh của gói miễn phí; bảng tạm được xoá sau khi đo.

6.7. Đánh giá tổng hợp

Bảng 6.6. Đánh giá mức đạt theo tiêu chí

Tiêu chí	Ngưỡng	Kết quả	Đánh giá
Đúng đắn dữ liệu	100%	23/23 bảng; 19/19 kiểm tra	Đạt
Nạp 16 bảng Analytics	< 120 s	72,66 s	Đạt
Truy vấn dashboard: trung vị / p95	< 300 / 500 ms	173,4 / 444,3 ms	Đạt
Tab dashboard cold (trung vị)	< 5 s	5,24 s; 5/7 tab vượt 5 s, chậm nhất 12,66 s	Chưa đạt
Tab dashboard có cache (trung vị)	< 2 s	1,04 s	Đạt
Bộ lọc native	100%	6/6 kịch bản	Đạt
View tổng hợp khi dữ liệu ×10	Không tăng	1,1 ms ở mọi hệ số	Đạt

Về RQ2, tầng phục vụ tái tạo chính xác tuyệt đối kết quả Spark và hai lớp dữ liệu nhất quán. Về RQ3, cả thời gian nạp lẫn độ trễ đều bị chi phối bởi mạng và kết nối chứ không bởi cơ sở dữ liệu; trải nghiệm cold chưa đạt ngưỡng 5 s (5/7 tab vượt) do mỗi chart mở kết nối TLS mới; cache khắc phục cho người dùng thường xuyên, và hướng cải tiến (pool kết nối, đặt Superset cùng vùng, làm nóng cache định kỳ) được nêu ở Mục 8.3.

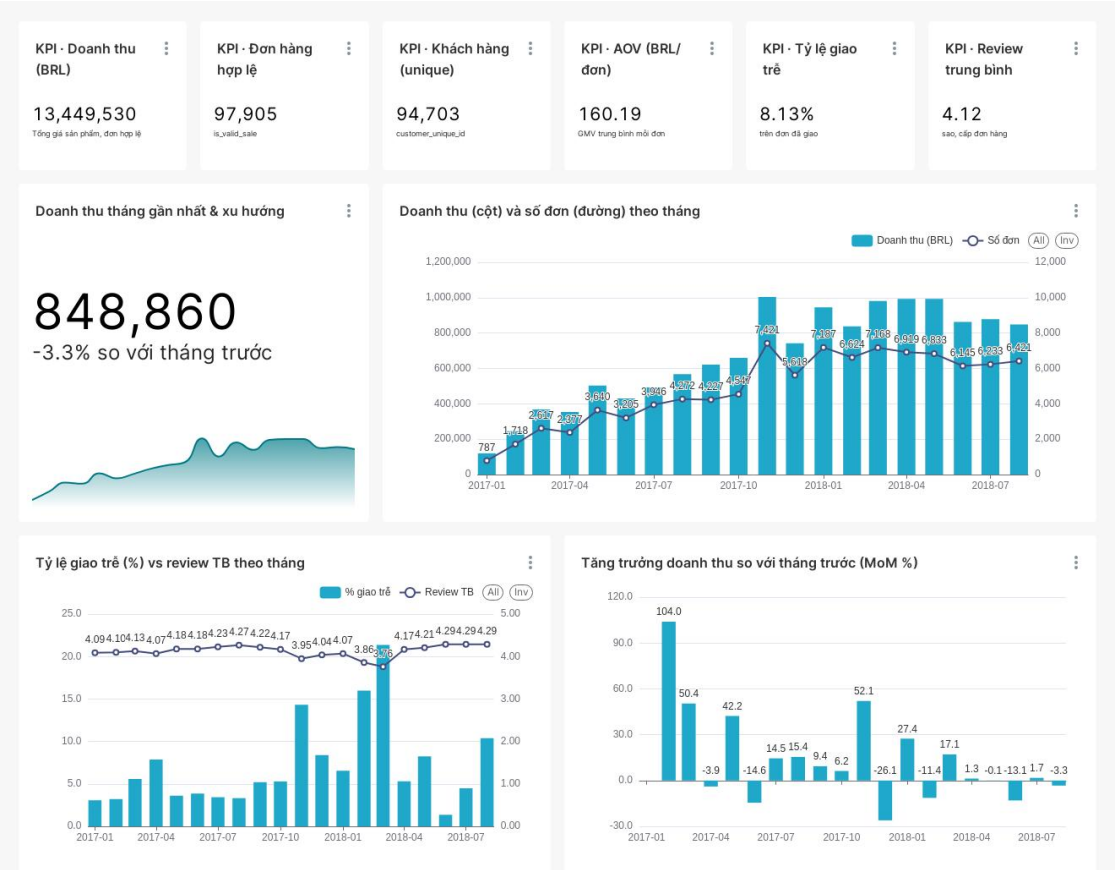
7. Kết quả demo

7.1. Luồng demo từ đầu vào đến đầu ra

1. **Đầu vào:** `data/analytics/` (16 bảng) và `data/gold/` (7 bảng) — Parquet do nhóm Analytics bàn giao.
2. **Nạp:** `docker compose run --rm tools python loader/load_to_neon.py` — log từng bảng, dòng tổng, `analytics.load_log` có `run_id` mới; các view được tạo lại.
3. **Kiểm chứng:** `validate.py` — 23/23 bảng khớp số dòng, 19/19 số neo "ĐẠT".
4. **Dashboard:** `docker compose up -d superset-db superset` rồi `build_dashboard.py` — mở `http://localhost:8088/superset/dashboard/olist-presentation/`, đi qua 7 tab, áp dụng bộ lọc.
5. **Đầu ra cho Decision/Action:** truy vấn `v_decision_hints` và `v_seller_watchlist` (Mục 7.4).

Các ảnh dưới đây được Playwright chụp tự động từ hệ thống đang chạy (toàn bộ 7 tab và 6 kịch bản lọc ở `evidence/`; một số tab ở Phụ lục E); để dễ đọc, báo cáo cắt phần chính của mỗi ảnh.

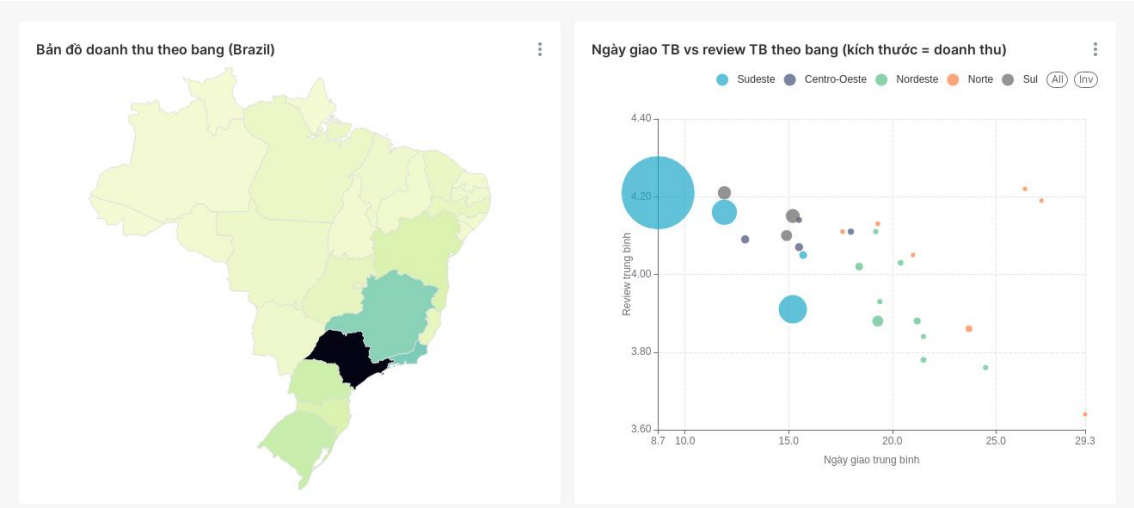
7.2. Dashboard và nội dung phân tích



Hình 7.1. *Tab 1 "Tổng quan & xu hướng": 6 thẻ KPI khung 2017-01 → 2018-08, doanh thu tháng gần nhất, doanh thu & số đơn theo tháng, % trễ vs review, MoM*

Tổng quan. Sáu thẻ KPI tính trên `v_orders`: doanh thu 13.449.530 BRL, 97.905 đơn hợp lệ, 94.703 khách, AOV 160,19 BRL, tỷ lệ trễ 8,13%, review 4,12. Doanh thu tăng từ ~120 nghìn BRL (01/2017) lên đỉnh **1.003.862 BRL vào 11/2017** (Black Friday, MoM +52,06%). Thông điệp cho ban điều hành: **bán nhiều không đồng nghĩa vận hành tốt** — 11/2017 trễ 14,31%, review 3,95; 03/2018 trễ 21,36%, review 3,76; trong khi 04–05/2018 doanh thu gần bằng đỉnh mà trễ chỉ 5–8%.

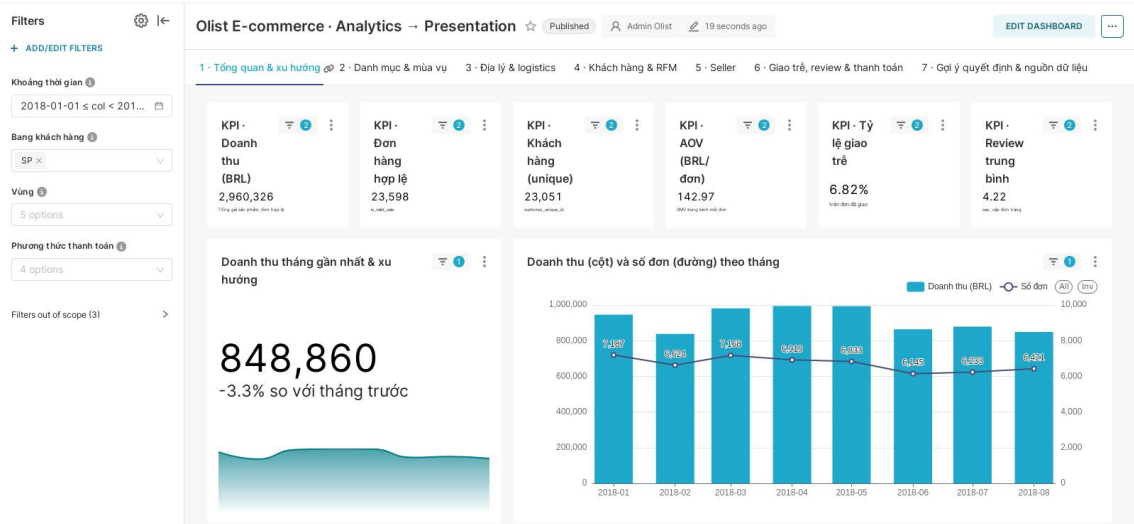
Danh mục (tab 2). `health_beauty` 9,30%, `watches_gifts` 8,88%, `bed_bath_table` 7,70% doanh thu; 17 danh mục lớn nhất mới chiếm 79,7% — "đuôi dài" của sàn đa ngành. Mọi ngành lớn đạt đỉnh tháng 11, nên tồn kho và quảng cáo cần dồn vào quý IV.



Hình 7.2. *Tab 3 "Địa lý & logistics": bản đồ doanh thu theo bang và quan hệ ngày giao – review (kích thước = doanh thu)*

Địa lý. SP chiếm 38,27% doanh thu, giao 8,7 ngày, trễ 5,89%, review 4,21 — góc "nhanh và hài lòng" vì gần người bán; BA (19,3 ngày, trễ 14,04%, review 3,88) và RJ (15,2 ngày, 13,47%, 3,91) ở góc ngược lại dù giá trị đơn cao hơn. Từ 0–200 km lên trên 1.000 km, phí ship tăng 12,66 → 31,65 BRL, ngày giao 7,0 → 19,0, tỷ lệ trễ 6,34% → 11,69% — ưu tiên kho/người bán theo vùng.

Khách hàng (tab 4). 96,96% khách chỉ mua một lần; 2.887 khách (3,04%) mua lại, chi trọn đời 308 so với 161 BRL. Phân khúc theo luật: 251 Champions, 675 Loyal, 1.961 At Risk; K-Means tự tách đúng 2.887 khách mua lại thành một cụm. **Giao trễ (tab 6).** Đơn trễ 2,56 sao so với 4,29; trễ 1–3 ngày còn 3,29 sao, trễ 4–7 ngày còn 2,10 sao với 58,6% một sao — căn cứ cho luật "can thiệp trước mốc 4 ngày".



Hình 7.3. *Bộ lọc native: khoảng thời gian 2018 + bang SP — thẻ KPI và biểu đồ tháng chỉ còn 8 tháng 2018 của bang SP (doanh thu 2.960.326 BRL, 23.598 đơn, trễ 6,82%)*

7.3. Cảnh báo người bán và gợi ý quyết định

Seller watchlist – cảnh báo vận hành (153 seller)

Search 153 records...

Seller	Bang seller	Mức cảnh báo	Số vi phạm	Lý do	Số đơn	Doanh thu (BRL)	Review TB	Ngày giao TB	% giao trễ	Số tháng hoạt động	Hành động đề xuất
04308b1e...	SC	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	94	60,131	3.88	18.9	11.96	9	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
522620dc...	PR	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	174	57,168	3.9	15.2	10.11	20	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
81602554...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	385	47,018	3.83	16.7	17.92	11	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
2eb70248...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	202	42,629	2.72	17.9	14.21	6	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
712e6ed8...	SC	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	79	39,815	3.45	23.7	21.18	14	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
c3867b46...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	240	36,486	3.88	15.1	11.2	16	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
855668e0...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	310	31,918	3.77	16.1	13.83	16	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
88460e8e...	PR	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	247	31,397	3.36	18.2	19.67	9	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày
b1b39487...	SP	ĐỎ	3	review < 4, giao > 15 ngày, trễ > 10%	16	22,249	1.81	23.4	64.29	3	Cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày

Hình 7.4. *Tab 5: bảng watchlist cảnh báo 153 người bán (số vi phạm, lý do, mức ĐỎ/VÀNG, hành động đề xuất)*

Chỉ 539/3.053 người bán (17,65%) tạo 80% doanh thu. Bảng watchlist liệt kê 153 người bán thuộc top 10% doanh thu nhưng vận hành kém; 75 người ở mức ĐỎ (vi phạm ≥ 2 tiêu chí) kèm hành động "cảnh cáo + giảm hiển thị; yêu cầu kế hoạch SLA 30 ngày". Ví dụ người bán `b1b39487…` (SP): review 1,81, giao 23,4 ngày, trễ 64,29%.

Gợi ý quyết định (luật + ngưỡng + hành động)

Search 7 records...

Ưu tiên	Mức độ	Chủ đề	Tín hiệu	Giá trị	Chi tiết	Ngưỡng	Hành động gợi ý
1	CAO	Seller	Seller doanh thu top 10% vi phạm ≥2 tiêu chí vận hành	75.00	75/153 seller trong watchlist, doanh thu watchlist 4,400,613 BRL	review < 4 | giao > 15 ngày | trễ > 10% (≥2 điều kiện)	Gửi cảnh cáo, giảm hiển thị, yêu cầu kế hoạch SLA 30 ngày
2	CAO	Giao hàng	Đơn trễ kéo review xuống	1.73	Review đúng hạn 4.29 vs trễ 2.56 (7655 đơn trễ)	chênh lệch > 1 sao	Cảnh báo seller khi đơn trễ 1–3 ngày (review 3,29) trước mốc 4–7 ngày (2,10)
3	CAO	Vận hành theo tháng	Tháng có tỷ lệ giao trễ > 15%	2.00	2018-03 (21.36%), 2018-02 (15.99%)	late_delivery_pct > 15%	Tăng năng lực vận chuyển trước cao điểm (T11 Black Friday, T2–T3)
4	TRUNG	Địa lý	Bang trễ	9.00	AL 23.93%/3.76, MA 19.67%/3.78,	late_delivery_pct > 10%	Seller/kho vùng;

Hình 7.5. *Tab 7: bảng "Gợi ý quyết định" — luật, giá trị hiện tại, ngưỡng và hành động (nguồn v_decision_hints)*

7.4. Đầu ra cho giai đoạn Decision/Action

Giai đoạn sau chỉ cần hai truy vấn để lấy toàn bộ đầu vào:

```
SELECT priority, severity, topic, signal, metric_value, threshold, action, source
FROM   presentation.v_decision_hints ORDER BY priority;          -- 7 luật

SELECT seller_id, seller_state, revenue, violations, reasons, recommended_action
FROM   presentation.v_seller_watchlist
WHERE  alert_level = 'ĐỎ' ORDER BY revenue DESC;   -- 75 người bán
```

Kết quả là dữ liệu có cấu trúc mà một rule engine có thể chuyển thành email/Telegram cảnh báo hoặc kích hoạt chăm sóc khách hàng khi đơn có nguy cơ trễ ≥ 4 ngày. Dashboard phục vụ con người, view phục vụ máy — hai kênh dùng chung một tầng ngữ nghĩa nên không mâu thuẫn số liệu.

8. Thảo luận

8.1. Điểm mạnh

- **Ranh giới rõ ràng, data contract tường minh:** nhóm phục vụ chứ không tính lại phân tích của Analytics; mọi con số truy ngược được về một cột Parquet và được đối chiếu tự động. Nhóm trước đổi thuật toán, nhóm sau chỉ cần nạp lại.
- **Tách tầng vật lý và ngữ nghĩa:** `analytics/gold` phản ánh Parquet, `presentation` phản ánh nghiệp vụ; định nghĩa KPI nằm ở một nơi, dùng chung cho dashboard và Decision/Action.
- **Độ đo không cộng dồn và lọc chéo nhất quán:** AOV, % trễ, review trên thẻ KPI tính lại từ `v_orders` thay vì trung bình của tỷ lệ tháng; kiểm tra nhất quán chéo chứng minh tổng từ gold bằng đúng tổng của Analytics.
- **Đầu ra máy đọc được:** `v_decision_hints` chứa luật, ngưỡng, giá trị và hành động — giai đoạn sau dùng ngay bằng SQL.
- **Tái lập bằng mã:** Docker Compose, DDL tự sinh, dashboard dựng qua REST API, ảnh chụp tự động, báo cáo này cũng sinh bằng script.
- **Chi phí gần bằng không khi không dùng:** scale-to-zero cộng cache 10 phút của Superset.

8.2. Hạn chế

- **Insight phân tích không lọc chéo được:** Pareto, RFM, K-Means, watchlist là kết quả toàn kỳ của Spark; lọc theo tháng/bang đòi hỏi tính lại phân tích, vượt phạm vi nhóm. Phạm vi từng bộ lọc phải được cấu hình và giải thích cẩn thận.
- **Chi phí lớp gold:** 7 bảng gold chiếm khoảng 79% thời gian nạp chỉ để phục vụ lọc chéo cho một số thẻ KPI và xu hướng.
- **Thời gian nạp phụ thuộc đường truyền client:** thông lượng COPY (~0,32 MB/s) bị giới hạn bởi băng thông tải lên, không phải bởi Neon.
- **Lần mở dashboard đầu chậm:** Superset dùng NullPool nên mỗi chart mở kết nối TLS mới (~1,1 s); khi cold, trung vị một tab 5,24 s và 5/7 tab vượt 5 s; chỉ khi có cache mới còn ~1 s.

- **Giới hạn gói miễn phí Neon:** compute cố định 0,25 – 0,25 CU, 1 GiB/nhánh — thí nghiệm mở rộng dừng ở ×10; chưa kiểm chứng autoscaling nhiều CU và tải hàng nghìn người dùng đồng thời.
- **Minh chứng Neon:** Neon Console cần đăng nhập tương tác nên ảnh kiểm kê được sinh từ truy vấn catalog trên database (`neon_tables_inventory.png`) và thông tin project lấy qua Neon API, không phải ảnh chụp Console.
- **Full refresh, chưa nạp tăng dần;** cache filesystem và metadata trong volume Docker cục bộ chưa phù hợp chạy nhiều bản Superset; bảo mật mức cơ bản (chưa row-level security, tài khoản quản trị dùng cho demo).

8.3. Khả năng mở rộng

Khi dữ liệu nguồn tăng 10–100 lần (1–10 triệu đơn):

- **Bảng Analytics tăng không đều:** bảng theo tháng, danh mục, bang, band khoảng cách gần như không đổi; chỉ bảng cấp khách hàng và người bán tăng tuyến tính, vẫn trong khả năng PostgreSQL với chỉ mục. Thí nghiệm E6 xác nhận: ở ×10, truy vấn trên dữ liệu thô tăng lên 506,6 ms còn view tổng hợp giữ 1,1 ms. Lớp gold chi tiết tăng tuyến tính và là ứng viên đầu tiên cần thay bằng bảng tổng hợp nhẹ hoặc materialized view.
- **Kết nối:** bật pool kết nối cho Superset (thay NullPool) hoặc đặt Superset cùng vùng AWS với Neon sẽ loại bỏ phần lớn ~0,7 s/chart khi cold; làm nóng cache định kỳ bằng Celery beat giữ trải nghiệm ~1 s/tab.
- **Nạp dữ liệu:** nút thắt hiện tại là mạng client → Neon. Bước đầu là chạy loader cùng vùng mạng với Neon (máy ảo AWS Singapore hoặc job CI), nơi `COPY` thường đạt hàng chục đến hàng trăm nghìn dòng/giây; sau đó nạp song song nhiều bảng, nạp tăng dần theo phân vùng tháng, và với bảng rất lớn thì dùng declarative partitioning, nạp vào bảng tạm rồi hoán đổi để không khoá người đọc.
- **Truy vấn:** Neon autoscaling tăng tài nguyên theo chiều dọc; *read replica* tách tải BI khỏi tải nạp; view nặng chuyển thành *materialized view* làm mới sau mỗi lần nạp.
- **Superset:** mở rộng theo chiều ngang — nhiều web server sau cân bằng tải, metadata PostgreSQL, cache Redis, Celery worker cho truy vấn bất đồng bộ và báo cáo định kỳ.

- **Khi nào đổi công nghệ?** Nếu phải lọc chéo tự do trên fact hàng tỷ dòng, kho OLAP cột (ClickHouse, Druid, Pinot) hoặc Trino trên Iceberg/Delta phù hợp hơn. Nhờ SQLAlchemy và tầng ngữ nghĩa SQL chuẩn, việc thay tầng phục vụ chủ yếu là chuyển các view, không làm lại dashboard.

8.4. Bài học kỹ thuật và hướng cải tiến

Bài học: (1) tầng trình bày không chỉ "vẽ biểu đồ" — rủi ro lớn nhất nằm ở ngữ nghĩa số liệu (grain, độ đo không cộng dồn, phạm vi thời gian), nên data contract và đối chiếu tự động là bắt buộc; (2) với dịch vụ serverless, độ trễ người dùng cảm nhận chủ yếu đến từ mạng và cold start, cache ở tầng BI có tác dụng lớn hơn tối ưu SQL ở quy mô này; (3) PgBouncer chế độ transaction hợp với truy vấn ngắn của BI nhưng không hợp với DDL/COPY — phải phân biệt endpoint pooled và direct; (4) dashboard-as-code tốn công ban đầu nhưng trả lại bằng khả năng tái lập và quản lý phiên bản.

Hướng cải tiến:

- Đề xuất Analytics bổ sung cột tử số/mẫu số (đơn đã giao, đơn trễ, đơn có review) và bảng tổng hợp tháng × bang × danh mục × PTTT, để bỏ lớp gold chi tiết mà vẫn lọc chéo được.
- Nạp tăng dần theo `run_id`/phân vùng tháng, lưu lịch sử nhiều lần chạy để so sánh phiên bản phân tích.
- Điều phối toàn pipeline bằng Airflow/Dagster (Analytics xong → nạp → kiểm tra → làm mới view → làm nóng cache); quản lý tầng ngữ nghĩa bằng dbt.
- Bật Alerts & Reports của Superset (Celery + Redis) để gửi cảnh báo khi KPI vượt ngưỡng — cầu nối tự nhiên sang Decision/Action; thêm row-level security, role chỉ đọc, SSO.

9. Kết luận

Đồ án đã thực hiện giai đoạn **Analytics → Presentation** của pipeline dữ liệu lớn trên bộ dữ liệu Olist. Đối chiếu với mục tiêu ở Chương 1:

- **Nạp đúng và đủ:** 23 bảng (16 Analytics + 7 gold, 505.592 dòng) được nạp vào Neon bằng loader tự sinh DDL và `COPY`, có `load_log`; 23/23 bảng khớp số dòng, 19/19 kiểm tra số neo/nhất quán đạt.
- **Tầng ngữ nghĩa:** 22 view và bảng `dim_state` định nghĩa KPI thống nhất, xử lý đúng độ đo không cộng dồn qua lớp cấp đơn, bổ sung cột trình bày và mã hoá 7 luật gợi ý quyết định.
- **Dashboard:** Superset 7 tab, 45 chart cho năm nhóm người dùng, có thẻ KPI, xu hướng, Pareto, bản đồ Brazil, bảng chi tiết, 7 bộ lọc native (6/6 kịch bản kiểm thử đạt) và khu vực gợi ý quyết định; một tab mở trong 1,04 s khi có cache.
- **Đo lường:** nạp toàn bộ trong 351,34 giây (16 bảng Analytics trong 72,66 giây), nút thắt là băng thông của client; 21 truy vấn dashboard có trung vị 173,4 ms phía client nhưng chỉ 1,25 ms thực thi trên server — mạng chiếm phần lớn độ trễ, còn bảng tổng hợp sẵn nhanh hơn hơn 250 lần so với tính lại từ fact; ở dữ liệu ×10, view tổng hợp vẫn giữ 1,1 ms.
- **Tái lập:** Docker Compose, mã nguồn tại https://github.com/viettuong287/BigDataProject, dashboard dựng bằng REST API, báo cáo sinh bằng script.

Dashboard truyền tải năm thông điệp chính: (1) doanh thu đỉnh không đồng nghĩa vận hành khoẻ (11/2017, 03/2018 trễ cao, review thấp); (2) giao trễ làm mất ~1,7 sao, sau 4 ngày trễ phần lớn khách chấm 1 sao; (3) địa lý quyết định trải nghiệm — SP gần người bán nên nhanh và hài lòng; (4) 97% khách chỉ mua một lần, CRM nên tập trung vào 3% khách mua lại và khách mới; (5) ~18% người bán tạo 80% doanh thu, 153 người bán lớn nhưng vận hành kém là rủi ro lớn nhất phía cung.

Khả năng tích hợp: với giai đoạn trước, nhóm giao tiếp qua data contract trên Parquet — Analytics chạy lại thì chỉ cần chạy lại loader. Với giai đoạn sau, `v_decision_hints` và `v_seller_watchlist` là giao diện máy đọc được để kích hoạt cảnh báo, thông báo hoặc điều chỉnh nghiệp vụ mà không phụ thuộc dashboard. Cùng

các nhóm khác, đồ án góp phần hình thành pipeline hoàn chỉnh từ dữ liệu thô đến hành động.

Tài liệu tham khảo

[1] Olist, "Brazilian E-Commerce Public Dataset by Olist," Kaggle, 2018. [Trực tuyến]. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[3] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[4] M. Armbrust et al., "Spark SQL: Relational Data Processing in Spark," in *Proc. ACM SIGMOD*, 2015, pp. 1383–1394.

[5] The Apache Software Foundation, "Apache Parquet – File Format Documentation." [Trực tuyến]. https://parquet.apache.org/docs/

[6] S. Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets," *Proc. VLDB Endowment*, vol. 3, no. 1–2, pp. 330–339, 2010.

[7] D. J. Abadi, S. R. Madden and N. Hachem, "Column-Stores vs. Row-Stores: How Different Are They Really?," in *Proc. ACM SIGMOD*, 2008, pp. 967–980.

[8] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.

[9] M. Armbrust, A. Ghodsi, R. Xin and M. Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Proc. CIDR*, 2021.

[10] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.

[11] S. Chaudhuri and U. Dayal, "An Overview of Data Warehousing and OLAP Technology," *ACM SIGMOD Record*, vol. 26, no. 1, pp. 65–74, 1997.

[12] Neon, "Neon architecture overview," Neon Documentation. [Trực tuyến]. https://neon.com/docs/introduction/architecture-overview

[13] Neon, "Autoscaling", "Scale to Zero" và "Connection pooling," Neon Documentation. [Trực tuyến]. https://neon.com/docs/introduction/autoscaling ; https://neon.com/docs/introduction/scale-to-zero ; https://neon.com/docs/connect/connection-pooling

[14] The PostgreSQL Global Development Group, "PostgreSQL 18 Documentation – COPY." [Trực tuyến]. https://www.postgresql.org/docs/18/sql-copy.html

[15] The Apache Software Foundation, "Apache Superset Documentation." [Trực tuyến]. https://superset.apache.org/docs/intro

[16] B. Shneiderman, "The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations," in *Proc. IEEE Symposium on Visual Languages*, 1996, pp. 336–343.

[17] S. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Analytics Press, 2013.

[18] P. S. Fader, B. G. S. Hardie and K. L. Lee, "RFM and CLV: Using Iso-Value Curves for Customer Base Analysis," *Journal of Marketing Research*, vol. 42, no. 4, pp. 415–430, 2005.

[19] The Apache Software Foundation, "Apache Arrow – Python bindings (pyarrow)." [Trực tuyến]. https://arrow.apache.org/docs/python/

[20] D. Varrazzo, "Psycopg 3 – Using COPY TO and COPY FROM." [Trực tuyến]. https://www.psycopg.org/psycopg3/docs/basic/copy.html

[21] Docker Inc., "Docker Compose Documentation." [Trực tuyến]. https://docs.docker.com/compose/

[22] Nhóm Storage → Processing, "big-data: pipeline làm sạch và tầng gold Olist," GitHub. [Trực tuyến]. https://github.com/NguyenAlpha/big-data

[23] Nhóm Processing → Analytics, "Analytics – Olist E-commerce: 16 bảng Parquet, báo cáo và tài liệu bàn giao," repo upstream `Analytics-main` do nhóm Analytics cung cấp; bản sao tài liệu bàn giao trong `docs/upstream/` của repo nhóm.

[24] A. Verbitski et al., "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases," in *Proc. ACM SIGMOD*, 2017, pp. 1041–1052.

[25] Bộ môn, "Yêu cầu đồ án học phần Nhập môn Dữ liệu lớn," tài liệu học phần, 2026.

Phụ lục A. Hướng dẫn chạy

Yêu cầu: Docker Desktop (đã thử với Docker 29), Git, Internet, một database Neon (gói miễn phí đủ dùng). Không cần cài Python trên máy host.

```
git clone https://github.com/viettuong287/BigDataProject.git
cd BigDataProject
copy .env.example .env      # điền NEON_DATABASE_URL (endpoint pooled),
SUPERSET_SECRET_KEY, mật khẩu admin

# 1. Nạp dữ liệu vào Neon (~6 phút, phụ thuộc băng thông tải lên) và kiểm chứng
docker compose build tools
docker compose run --rm tools python loader/load_to_neon.py   # schema, COPY 23 bảng,
index, view
docker compose run --rm tools python loader/validate.py       # 23 bảng + 19 số neo
docker compose run --rm tools python loader/apply_sql.py      # (tuỳ chọn) chỉ tạo lại
view

# 2. Bật Superset (metadata Postgres 16) và dựng dashboard từ code (~4 phút lần đầu)
docker compose up -d superset-db superset
docker compose run --rm tools python superset/build_dashboard.py
#    mở http://localhost:8088/superset/dashboard/olist-presentation/  (admin / mật
khẩu trong .env)
#    dự phòng: docker compose run --rm tools python superset/import_dashboard.py

# 3. Tái tạo minh chứng (tuỳ chọn)
docker compose run --rm tools python scripts/measure_latency.py
docker compose run --rm tools python scripts/scale_experiment.py
docker compose run --rm tools python scripts/neon_inventory.py
docker exec olist-superset bash -c 'rm -rf /app/superset_home/cache/*'
docker compose run --rm screenshots                        # ảnh + thời gian tải
cold/cache
docker compose run --rm screenshots python scripts/render_diagrams.py

# 4. Sinh lại báo cáo (Python 3.12 trên máy hoặc trong Docker)
python -m venv report\.venv; report\.venv\Scripts\python -m pip install -r
report\requirements.txt
report\.venv\Scripts\python report\build_report.py
powershell -ExecutionPolicy Bypass -File report\tools\word_stats.ps1 -Pdf   # cập nhật
mục lục, xuất PDF
```

Tắt hệ thống: `docker compose down` (thêm `-v` sẽ xoá metadata Superset; dữ liệu trên Neon không bị ảnh hưởng). Nếu đang mở dashboard trong trình duyệt khi chạy lại `build_dashboard.py`, hãy tải lại trang để tránh ghi đè cấu hình bộ lọc. Trong Word, nhấn Ctrl+A rồi F9 để cập nhật mục lục và danh mục hình/bảng.

Phụ lục B. So sánh phương án công nghệ

Bảng B.1. So sánh công cụ trình bày (BI) cho giai đoạn Presentation

Tiêu chí	Apache Superset	Power BI	Tableau	Grafana	Metabase
Giấy phép	Mã nguồn mở (Apache 2.0)	Thương mại (chia sẻ cần Pro)	Thương mại	Mã nguồn mở (AGPL)	Mã nguồn mở / thương mại
Triển khai	Docker, tự host	Windows Desktop + cloud	Desktop + Server/Cloud	Docker	Docker
Thế mạnh	BI SQL-native, nhiều chart, native filter, REST API	Mô hình DAX, thân thiện	Trực quan hoá linh hoạt	Chuỗi thời gian, giám sát	Dễ dùng
Hạn chế	Cấu hình ban đầu phức tạp	Khó tái lập bằng mã	Chi phí bản quyền	Yếu với phân tích theo danh mục	Ít loại biểu đồ, bộ lọc hạn chế
Dashboard-as-code	Có (REST API, export YAML)	Hạn chế (.pbix nhị phân)	Hạn chế	Có (JSON)	Một phần

Bảng B.2. So sánh phương án tầng lưu trữ phục vụ

Tiêu chí	Neon Serverless Postgres	Postgres cục bộ (Docker)	ClickHouse	Đọc thẳng Parquet (DuckDB)
Mô hình lưu trữ	Hàng + chỉ mục	Hàng + chỉ mục	Cột (MergeTree)	Cột (tệp)
Phù hợp quy mô	Hàng chục–trăm triệu dòng tổng hợp	Giới hạn một máy	Hàng tỷ dòng chi tiết	Lớn, nhưng không phục vụ đồng thời
Chia sẻ cho nhóm/giảng viên	Có, qua Internet + TLS	Không	Cần tự host trên cloud	Phải chia sẻ tệp
Vận hành	Được quản lý, scale-to-zero	Tự quản lý	Tự quản lý / trả phí	Không có dịch vụ
Tương thích BI	Rất tốt (chuẩn Postgres)	Rất tốt	Tốt (driver riêng)	Trung bình

Phụ lục C. Data contract chi tiết

Bảng C.1. Ánh xạ kiểu dữ liệu Parquet → PostgreSQL do loader áp dụng

Kiểu Arrow/Parquet	Kiểu PostgreSQL	Lý do	Số bảng dùng
int32	INTEGER	Số nguyên 4 byte (tháng, điểm, rank)	8
int64	BIGINT	Đếm số lượng lớn (đơn, khách, món)	16
decimal128(p, s)	NUMERIC(p,s)	Tiền BRL: giữ chính xác 2 chữ số thập phân, không sai số dấu phẩy động	16
double	DOUBLE PRECISION	Giá trị trung bình/tỷ lệ đã làm tròn	10
bool	BOOLEAN	Cờ (vd. tháng Black Friday)	1
string	TEXT	Chuỗi định danh/nhãn, không giới hạn độ dài	14
date32[day]	DATE	Ngày mua đầu/cuối	2

Bảng C.2. Các bảng gold được nạp bổ sung vào schema gold (đầu vào chi tiết cho bộ lọc chéo)

Bảng	1 dòng =	Số dòng	Số cột	Số file	MB
gold.agg_category_monthly	1 danh mục x 1 tháng	1.282	9	1	0,03
gold.agg_customer_rfm	1 khách hàng thật	94.983	9	1	4,32
gold.agg_customer_state	1 bang của khách hàng	27	11	1	0,01
gold.agg_sales_monthly	1 tháng	24	12	1	0,01
gold.agg_seller_performance	1 người bán	3.053	15	1	0,20
gold.fact_order_items	1 sản phẩm trong 1 đơn hàng	112.650	27	24	16,93
gold.fact_orders	1 đơn hàng	99.441	29	25	13,17
Tổng		**311.460**		54	34,65

Bảng C.3. Schema 16 bảng Analytics trên Neon (sinh tự động từ Parquet)

Bảng analytics.*	Số dòng	Cột (kiểu PostgreSQL)
kpi_monthly	20	purchase_year (integer), purchase_month (integer), orders (bigint), customers (bigint), items (bigint), revenue (numeric(32,2)), freight (numeric(32,2)), gmv (numeric(33,2)), avg_order_value (numeric(24,2)), avg_review_score (double precision), avg_delivery_days (double precision), late_delivery_pct (double precision), revenue_mom_pct (numeric(35,2)), orders_mom_pct (numeric(27,2)), revenue_yoy_pct (numeric(35,2)), revenue_rank (integer), is_black_friday_month (boolean)
category_pareto	74	category (text), orders (bigint), items (bigint), revenue (numeric(32,2)), freight (numeric(32,2)), avg_price (numeric(33,2)), freight_to_revenue_pct (numeric(35,2)), revenue_share_pct (numeric(35,2)), cumulative_share_pct (numeric(35,2))
category_seasonality	715	category (text), purchase_month (integer), revenue (numeric(32,2)), orders (bigint), pct_of_category_year (numeric(35,2))
state_logistics	27	customer_state (text), orders (bigint), customers (bigint), revenue (numeric(32,2)), gmv (numeric(33,2)), avg_order_value (numeric(24,2)), avg_freight (numeric(23,2)), freight_pct_of_revenue (numeric(35,2)), avg_delivery_days (double precision), late_delivery_pct (double precision), avg_review_score (double precision), revenue_share_pct (numeric(35,2)), revenue_rank (integer)
insight_distance	4	distance_band (text), band_order (integer), items (bigint), avg_freight (numeric(13,2)), avg_delivery_days (double precision), avg_delay_days (double precision), late_pct (double precision)
rfm_scores	94.983	customer_unique_id (text), customer_state (text), first_purchase_date (date), last_purchase_date (date), recency_days (integer), frequency (bigint), monetary (numeric(33,2)), avg_order_value (numeric(24,2)), r_score (integer), f_score (integer), m_score (integer), rfm_segment (text)
customer_segments	94.983	customer_unique_id (text), customer_state (text), first_purchase_date (date), last_purchase_date (date), recency_days (integer), frequency (bigint), monetary (numeric(33,2)), avg_order_value (numeric(24,2)), r_score (integer), f_score (integer), m_score (integer), rfm_segment (text), cluster (integer)

Bảng analytics.*	Số dòng	Cột (kiểu PostgreSQL)
cluster_profile	4	cluster (integer), customers (bigint), avg_recency (double precision), avg_frequency (double precision), avg_monetary (numeric(34,2)), champions (bigint), loyal (bigint), at_risk (bigint)
repeat_profile	2	cohort (text), customers (bigint), pct (numeric(27,2)), avg_recency (double precision), avg_frequency (double precision), avg_monetary (numeric(34,2)), avg_aov (numeric(25,2))
repeat_categories	70	category (text), repeat_customers (bigint), items (bigint), revenue (numeric(23,2))
repeat_states	27	customer_state (text), repeat_customers (bigint), avg_frequency (double precision), avg_monetary (numeric(34,2)), avg_aov (numeric(25,2))
seller_pareto	3.053	seller_id (text), seller_state (text), orders (bigint), revenue (numeric(22,2)), avg_review_score (double precision), avg_delivery_days (double precision), late_delivery_pct (double precision), avg_distance_km (double precision), active_months (bigint), revenue_share_pct (numeric(29,2)), cumulative_share_pct (numeric(35,2))
seller_watchlist	153	seller_id (text), seller_state (text), orders (bigint), revenue (numeric(22,2)), avg_review_score (double precision), avg_delivery_days (double precision), late_delivery_pct (double precision), avg_distance_km (double precision), active_months (bigint), revenue_pctile (double precision)
insight_late_vs_review	2	delivery (text), orders (bigint), avg_review (double precision), pct_score_1_2 (numeric(8,2)), pct_has_comment (numeric(8,2))
delay_vs_review	5	delay_band (text), band_order (integer), orders (bigint), avg_review (double precision), pct_1_star (numeric(8,2)), pct_has_comment (numeric(8,2))
payment_mix	10	main_payment_type (text), installment_band (text), orders (bigint), gmv (numeric(34,2)), avg_order_value (numeric(24,2))

Phụ lục D. Danh mục đối tượng tầng ngữ nghĩa

Mã SQL đầy đủ: `sql/10_presentation_views.sql` (tầng ngữ nghĩa) và `sql/90_anchor_checks.sql` (19 kiểm tra số neo) trong repo.

Bảng D.1. Các đối tượng của tầng ngữ nghĩa: 22 view và 1 bảng dimension (quét tự động từ `sql/10_presentation_views.sql`)

Đối tượng (schema presentation)	Loại	Nguồn	Mục đích
dim_state	bảng	–	Dimension 27 bang: tên, vùng, mã ISO BR-xx cho Country Map
v_kpi_monthly	view	analytics.kpi_monthly	Chuỗi KPI tháng; thêm month_date (DATE) và month_label cho bộ lọc thời gian
v_orders	view	gold.fact_orders, presentation.dim_state	Đơn hợp lệ cấp đơn 2017-01→2018-08: thẻ KPI lọc được theo tháng/bang/PTTT; late_flag cho % trễ chính xác
v_kpi_overview	view	analytics.insight_late_vs_review, analytics.kpi_monthly, analytics.repeat_profile, analytics.rfm_scores, analytics.seller_pareto, analytics.seller_watchlist, gold.agg_sales_monthly, presentation.v_kpi_monthly	1 dòng số neo toàn kỳ (doanh thu gold, tháng đỉnh, review trễ/đúng hạn, % khách lặp, watchlist)
v_category_pareto	view	analytics.category_pareto	Pareto danh mục + hạng + nhóm A/B/C
v_category_seasonality	view	analytics.category_seasonality, presentation.v_category_pareto	Heatmap danh mục × tháng năm 2017
v_category_monthly	view	gold.agg_category_monthly	Doanh thu danh mục theo tháng (từ gold) cho xu hướng có lọc danh mục
v_sales_items	view	gold.fact_order_items, presentation.dim_state	Dòng sản phẩm hợp lệ: doanh thu/ship theo danh mục × bang × tháng (không chứa review)
v_state_logistics	view	analytics.state_logistics, presentation.dim_state	Logistics theo bang, kèm tên bang, vùng, mã ISO
v_insight_distance	view	analytics.insight_distance	Phí ship, ngày giao, % trễ theo band khoảng cách
v_rfm_customers	view	analytics.customer_segments, presentation.dim_state	1 khách: điểm RFM, phân khúc, cohort, cụm K-Means có tên nghiệp vụ
v_rfm_segments	view	presentation.v_rfm_customers	Tổng hợp theo phân khúc RFM + hành động CRM gợi ý
v_cluster_profile	view	analytics.cluster_profile	Hồ sơ 4 cụm K-Means với tên nghiệp vụ
v_repeat_profile	view	analytics.repeat_profile	Khách mua lại vs mua một lần
v_repeat_categories	view	analytics.repeat_categories	Danh mục ưa thích của khách mua lại
v_repeat_states	view	analytics.repeat_states, presentation.dim_state	Bang của khách mua lại
v_seller_pareto	view	analytics.seller_pareto, analytics.seller_watchlist	Pareto người bán, nhóm A/B/C, cờ thuộc watchlist
v_seller_watchlist	view	analytics.seller_watchlist	Người bán cần theo dõi: số vi phạm, lý do, mức cảnh báo ĐỎ/VÀNG, hành động khuyến nghị
v_late_vs_review	view	analytics.insight_late_vs_review	Review đơn trễ vs đúng hạn (nhãn tiếng Việt)
v_delay_vs_review	view	analytics.delay_vs_review	Review và % 1 sao theo số ngày trễ
v_payment_mix	view	analytics.payment_mix	Cơ cấu PTTT × kỳ trả góp, % đơn

Đối tượng (schema presentation)	Loại	Nguồn	Mục đích
v_decision_hints	view	analytics.category_pareto, analytics.delay_vs_review, analytics.insight_late_vs_review, analytics.seller_pareto, analytics.state_logistics, presentation.v_category_pareto, presentation.v_kpi_monthly, presentation.v_rfm_segments, presentation.v_seller_pareto, presentation.v_seller_watchlist, presentation.v_state_logistics	7 luật gợi ý quyết định có ngưỡng, giá trị, hành động, nguồn – giao diện cho Decision/Action
v_load_log	view	analytics.load_log	Lần nạp gần nhất của mỗi bảng (lineage, độ tươi dữ liệu trên dashboard)

Phụ lục E. Minh chứng thực nghiệm chi tiết

E.1. Kết quả nạp từng bảng

Bảng E.1. Kết quả nạp từng bảng, lần chạy `20261112-145508-40efaf` (tệp `evidence/load_log.csv`)

Bảng	Số dòng	Parquet (MB)	CSV truyền (MB)	COPY (s)	PK/index + ANALYZE (s)	Tổng (s)	MB/s COPY	Khớp
analytics.kpi_monthly	20	0,01	0,00	0,79	0,68	1,49	0,00	✓
analytics.category_pareto	74	0,01	0,01	1,08	0,60	1,70	0,01	✓
analytics.category_seasonality	715	0,01	0,03	1,04	0,71	1,76	0,03	✓
analytics.state_logistics	27	0,01	0,00	0,88	0,53	1,42	0,00	✓
analytics.insight_distance	4	0,00	0,00	0,96	0,56	1,54	0,00	✓
analytics.rfm_scores	94.983	3,99	11,49	18,39	1,14	19,63	0,62	✓
analytics.repeat_profile	2	0,00	0,00	0,93	0,58	1,53	0,00	✓
analytics.repeat_categories	70	0,00	0,00	0,97	0,59	1,58	0,00	✓
analytics.repeat_states	27	0,00	0,00	0,84	0,70	1,55	0,00	✓
analytics.customer_segments	94.983	4,01	11,87	26,73	1,36	28,19	0,44	✓
analytics.cluster_profile	4	0,00	0,00	0,81	0,60	1,42	0,00	✓
analytics.seller_pareto	3.053	0,16	0,30	1,28	0,83	2,13	0,23	✓
analytics.seller_watchlist	153	0,01	0,02	0,84	0,70	1,56	0,02	✓
analytics.insight_late_vs_review	2	0,00	0,00	1,03	0,58	1,63	0,00	✓
analytics.delay_vs_review	5	0,00	0,00	3,38	0,59	3,99	0,00	✓
analytics.payment_mix	10	0,00	0,00	0,91	0,63	1,55	0,00	✓
gold.fact_orders	99.441	13,17	31,71	101,08	1,46	102,97	0,31	✓
gold.fact_order_items	112.650	16,93	42,19	154,98	2,83	158,29	0,27	✓
gold.agg_sales_monthly	24	0,01	0,00	0,93	0,58	1,53	0,00	✓
gold.agg_category_monthly	1.282	0,03	0,09	0,97	0,52	1,50	0,10	✓
gold.agg_customer_state	27	0,01	0,00	1,09	0,54	1,65	0,00	✓
gold.agg_seller_performance	3.053	0,20	0,43	1,20	0,55	1,77	0,35	✓
gold.agg_customer_rfm	94.983	4,32	9,21	9,70	1,18	10,96	0,95	✓
Tổng	**505.592**	42,88	107,36	330,80	19,06	**351,34**	0,32	

E.2. Kiểm tra số neo và nhất quán

Bảng E.2. Đối chiếu số neo bàn giao của Analytics và tính nhất quán giữa các view trên Neon

Kiểm tra (sql/90_anchor_checks.sql)	Kỳ vọng	Thực tế trên Neon	Kết quả	ms
Gold SUM(revenue) – agg_sales_monthly	13.494.400,74	13.494.400,74	ĐẠT	82,2
Gold SUM(price) – fact_order_items (is_valid_sale)	13.494.400,74	13.494.400,74	ĐẠT	296,3
kpi_monthly có 20 tháng	20	20	ĐẠT	76,2
kpi_monthly không chứa 2016 và 2018-09	0	0	ĐẠT	100,7
Tổng revenue kpi_monthly (nhỏ hơn gold vì cắt tháng rỗng)	info	13.449.529,68	INFO	100,9
Tháng đỉnh doanh thu	2017-11	2017-11	ĐẠT	122,4
MoM tháng đỉnh (%)	52,06	52,06	ĐẠT	111,0
Review đơn trễ	2,56	2,56	ĐẠT	117,8
Review đơn đúng hạn	4,29	4,29	ĐẠT	84,3
Số khách mua lại (frequency ≥ 2)	2.887	2.887	ĐẠT	169,8
Tổng khách thật (customer_unique_id)	94.983	94.983	ĐẠT	226,8
Tỷ lệ khách mua lại (%)	3,04	3,04	ĐẠT	109,3
Tỷ lệ frequency = 1 (%)	96,96	96,96	ĐẠT	184,8
Seller watchlist	153	153	ĐẠT	100,8
Số seller tạo ≤80% doanh thu	539	539	ĐẠT	186,5
Nhất quán: SUM(revenue) v_orders = SUM(revenue) kpi_monthly	0	0	ĐẠT	349,1

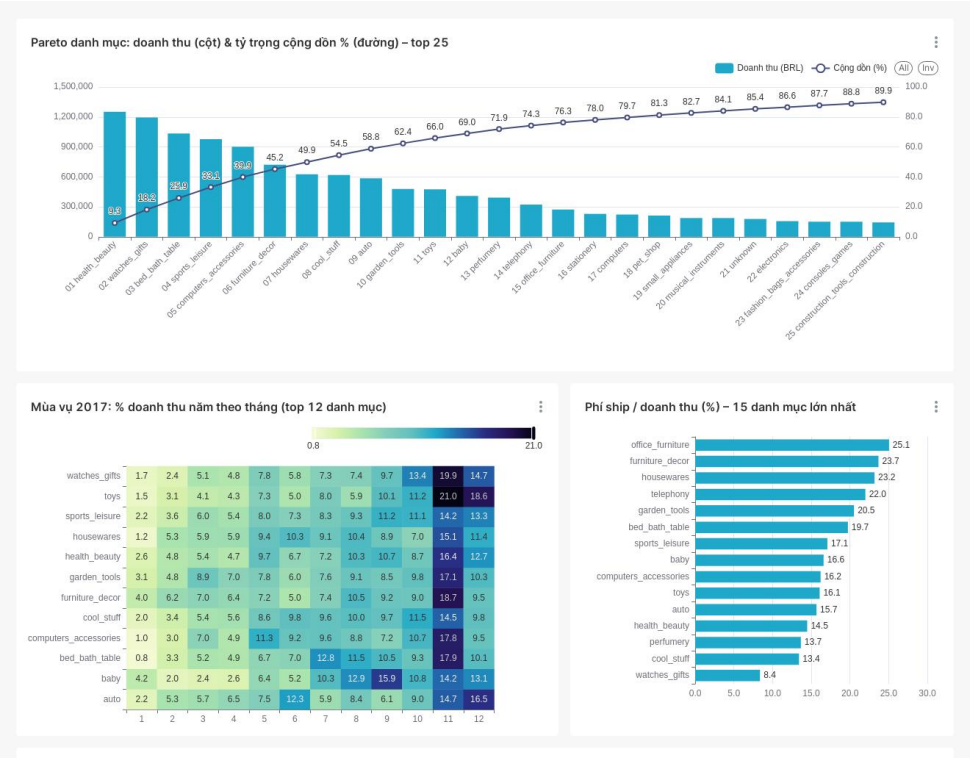
Kiểm tra (sql/90_anchor_checks.sql)	Kỳ vọng	Thực tế trên Neon	Kết quả	ms
Nhất quán: COUNT đơn v_orders = SUM(orders) kpi_monthly	0	0	ĐẠT	131,9
Nhất quán: SUM(revenue) v_sales_items = SUM(revenue) kpi_monthly	0	0	ĐẠT	166,4
Nhất quán: tổng khách theo segment = tổng khách RFM	94.983	94.983	ĐẠT	186,6
Số dòng gợi ý quyết định	7	7	ĐẠT	160,1

E.3. Độ trễ 22 truy vấn dashboard

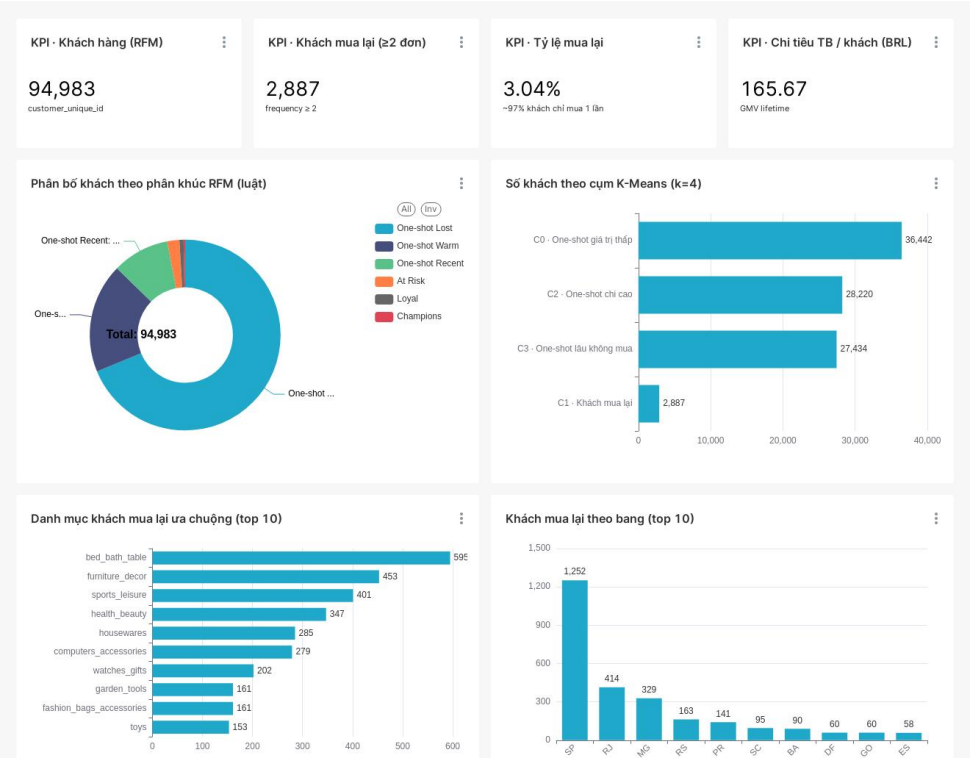
Bảng E.3. Độ trễ truy vấn các view trên Neon (22/22 phép đo thành công; tệp `evidence/query_latency.csv`)

Tab	Truy vấn	Số dòng	Client trung vị (ms)	Client p95 (ms)	Server thực thi (ms)
Tổng quan	Thẻ KPI (6 chỉ số) – v_orders	1	244.0	427.3	167.87
Tổng quan	Thẻ KPI có lọc bang SP + 2018	1	151.0	442.0	50.27
Tổng quan	Xu hướng tháng – v_kpi_monthly	20	174.4	255.5	0.16
Danh mục	Pareto top 25 – v_category_pareto	25	129.6	235.5	0.24
Danh mục	Heatmap mùa vụ – v_category_seasonality	144	207.5	425.4	0.48
Danh mục	Xu hướng danh mục – v_category_monthly	1246	287.6	352.6	1.48
Danh mục	Doanh thu danh mục × bang (lọc 3 danh mục) – v_sales_items	81	171.0	329.4	53.0
Địa lý	Bản đồ / bảng bang – v_state_logistics	27	129.8	226.0	0.22
Địa lý	Khoảng cách – v_insight_distance	4	126.0	219.2	0.17
Khách hàng	Pie phân khúc RFM – v_rfm_customers (94.983 dòng)	6	189.0	301.2	95.88
Khách hàng	Cụm K-Means lọc bang RJ + 3 segment	1	173.4	226.4	1.25
Khách hàng	Bảng segment – v_rfm_segments	6	247.0	374.1	151.95
Seller	Pareto nhóm seller – v_seller_pareto	3	201.7	302.7	2.36
Seller	Bubble top 300 seller – v_seller_pareto	300	129.6	443.9	2.5
Seller	Bảng watchlist – v_seller_watchlist	153	151.7	444.3	0.51
Review/PTTT	Trễ vs review – v_late_vs_review	2	131.5	254.5	0.11
Review/PTTT	Ngưỡng ngày trễ – v_delay_vs_review	5	99.7	234.0	0.11
Review/PTTT	Payment mix – v_payment_mix	10	108.1	206.6	0.14
Review/PTTT	Pie PTTT – v_orders	4	192.1	405.8	117.72
Quyết định	Gợi ý quyết định – v_decision_hints	7	189.7	282.3	108.16
Quyết định	Nhật ký nạp – v_load_log	23	186.4	197.8	0.18
So sánh	Pareto danh mục tính lại từ gold.fact_order_items (không dùng bảng tổng hợp)	25	171.4	337.4	66.43

E.4. Ảnh chụp bổ sung



Hình E.1. *Tab 2 "Danh mục & mùa vụ": Pareto top 25, heatmap mùa vụ 2017, tỷ lệ ship/doanh thu*



Hình E.2. *Tab 4 "Khách hàng & RFM": thẻ khách hàng, phân khúc RFM, cụm K-Means*

Neon Postgres · project `bigdata-olist-presentation` (muddy-wave-64763270)

Sinh bởi scripts/neon_inventory.py lúc 2026-11-12 23:45 từ catalog của database **olist_analytics** · PostgreSQL 18.6 (ac70818)

aws-ap-southeast-1	PostgreSQL 18	0.25 – 0.25 CU	146 MB	25 bảng · 22 view	free_v3
Region	Phiên bản	Compute (autoscaling)	Dung lượng database	analytics / gold / presentation	Gói Neon

- **analytics.*** – 17 table, 194,155 dòng, 39.9 MB
- **gold.*** – 7 table, 311,460 dòng, 98.0 MB
- **presentation.*** – 1 table, 27 dòng, 0.0 MB
- **presentation.*** – 22 view, 310,089 dòng

Schema	Đối tượng	Loại	Số dòng	MB	Index	Mô tả (COMMENT)
analytics	**category_pareto**	table	74	0.03	1	Grain: 1 danh mục. Nguồn: data/analytics/category_pareto (Parquet)
analytics	**category_seasonality**	table	715	0.11	1	Grain: 1 danh mục × 1 tháng năm 2017. Nguồn: data/analytics/category_s
analytics	**cluster_profile**	table	4	0.02	1	Grain: 1 cụm K-Means. Nguồn: data/analytics/cluster_profile (Parquet)
analytics	**customer_segments**	table	94,983	19.92	4	Grain: 1 khách thật + cụm K-Means. Nguồn: data/analytics/customer_segm
analytics	**delay_vs_review**	table	5	0.03	1	Grain: 1 band ngày trễ. Nguồn: data/analytics/delay_vs_review (Parquet
analytics	**insight_distance**	table	4	0.03	1	Grain: 1 band khoảng cách. Nguồn: data/analytics/insight_distance (Par
analytics	**insight_late_vs_review**	table	2	0.03	1	Grain: late vs on_time. Nguồn: data/analytics/insight_late_vs_review (
analytics	**kpi_monthly**	table	20	0.02	1	Grain: 1 tháng (2017-01→2018-08). Nguồn: data/analytics/kpi_monthly (P
analytics	**load_log**	table	23	0.03	1	
analytics	**payment_mix**	table	10	0.03	1	Grain: PTTT × band trả góp. Nguồn: data/analytics/payment_mix (Parquet
analytics	**repeat_categories**	table	70	0.03	1	Grain: 1 danh mục của khách lặp. Nguồn: data/analytics/repeat_categori
analytics	**repeat_profile**	table	2	0.03	1	Grain: repeat vs one_shot. Nguồn: data/analytics/repeat_profile (Parqu
analytics	**repeat_states**	table	27	0.03	1	Grain: 1 bang của khách lặp. Nguồn: data/analytics/repeat_states (Parq
analytics	**rfm_scores**	table	94,983	18.81	3	Grain: 1 khách thật. Nguồn: data/analytics/rfm_scores (Parquet)
analytics	**seller_pareto**	table	3,053	0.68	2	Grain: 1 seller. Nguồn: data/analytics/seller_pareto (Parquet)
analytics	**seller_watchlist**	table	153	0.06	1	Grain: seller top 10% doanh thu có vận hành kém. Nguồn: data/analytics
analytics	**state_logistics**	table	27	0.03	1	Grain: 1 bang khách hàng. Nguồn: data/analytics/state_logistics (Parqu
gold	**agg_category_monthly**	table	1,282	0.24	1	Grain: 1 danh mục × 1 tháng. Nguồn: data/gold/agg_category_monthly (Pa
gold	**agg_customer_rfm**	table	94,983	15.54	1	Grain: 1 khách thật. Nguồn: data/gold/agg_customer_rfm (Parquet)
gold	**agg_customer_state**	table	27	0.03	1	Grain: 1 bang khách hàng. Nguồn: data/gold/agg_customer_state (Parquet
gold	**agg_sales_monthly**	table	24	0.02	1	Grain: 1 tháng (gồm 2016, 2018-09). Nguồn: data/gold/agg_sales_monthly
gold	**agg_seller_performance**	table	3,053	0.72	1	Grain: 1 seller. Nguồn: data/gold/agg_seller_performance (Parquet)
gold	**fact_order_items**	table	112,650	46.71	5	Grain: 1 sản phẩm trong 1 đơn. Nguồn: data/gold/fact_order_items (Parq
gold	**fact_orders**	table	99,441	34.71	4	Grain: 1 đơn hàng. Nguồn: data/gold/fact_orders (Parquet)
presentation	**dim_state**	table	27	0.03	1	
presentation	**v_category_monthly**	view	1,246			
presentation	**v_category_pareto**	view	74			
presentation	**v_category_seasonality**	view	715			
presentation	**v_cluster_profile**	view	4			
presentation	**v_decision_hints**	view	7			
presentation	**v_delay_vs_review**	view	5			
presentation	**v_insight_distance**	view	4			
presentation	**v_kpi_monthly**	view	20			
presentation	**v_kpi_overview**	view	1			
presentation	**v_late_vs_review**	view	2			
presentation	**v_load_log**	view	23			
presentation	**v_orders**	view	97,905			
presentation	**v_payment_mix**	view	10			
presentation	**v_repeat_categories**	view	70			
presentation	**v_repeat_profile**	view	2			
presentation	**v_repeat_states**	view	27			
presentation	**v_rfm_customers**	view	94,983			
presentation	**v_rfm_segments**	view	6			
presentation	**v_sales_items**	view	111,752			
presentation	**v_seller_pareto**	view	3,053			
presentation	**v_seller_watchlist**	view	153			
presentation	**v_state_logistics**	view	27			

Hình E.3. *Kiểm kê Neon sinh từ catalog database: 25 bảng, 22 view, số dòng, dung lượng, 0,25 CU, PostgreSQL 18*

E.5. Danh sách tệp minh chứng

Bảng E.4. Danh sách tệp minh chứng trong evidence/

Tệp	Loại	KB
`evidence/apply_sql.log`	LOG	1,7
`evidence/architecture_pipeline.png`	Ảnh	55,7
`evidence/dashboard_load_latency.md`	MD	2,5
`evidence/dashboard_load_summary.json`	JSON	0,3
`evidence/dashboard_load_time.csv`	CSV	0,6
`evidence/dashboard_tab1_tong_quan_xu_huong.png`	Ảnh	240,0
`evidence/dashboard_tab2_danh_muc_mua_vu.png`	Ảnh	342,1
`evidence/dashboard_tab3_dia_ly_logistics.png`	Ảnh	238,5
`evidence/dashboard_tab4_khach_hang_rfm.png`	Ảnh	222,0
`evidence/dashboard_tab5_seller.png`	Ảnh	52,8
`evidence/dashboard_tab6_giao_tre_review_thanh_toan.png`	Ảnh	204,0
`evidence/dashboard_tab7_goi_y_quyet_dinh_nguon_du_lieu.png`	Ảnh	262,2
`evidence/filter_tab1_thoi_gian_2018_bang_SP.png`	Ảnh	205,2
`evidence/filter_tab2_danh_muc_bed_bath_health_watches.png`	Ảnh	236,9
`evidence/filter_tab3_vung_nordeste.png`	Ảnh	234,6
`evidence/filter_tab4_rfm_khach_mua_lai_bang_RJ.png`	Ảnh	242,0
`evidence/filter_tab5_bang_seller_SP.png`	Ảnh	229,5
`evidence/filter_tab6_pttt_boleto.png`	Ảnh	241,5
`evidence/load_log.csv`	CSV	3,2
`evidence/load_run.log`	LOG	4,1
`evidence/neon_inventory.csv`	CSV	3,8

Tệp	Loại	KB
evidence/neon_inventory.html	HTML	11,3
evidence/neon_project.json	JSON	0,5
evidence/neon_tables_inventory.png	Ảnh	301,8
evidence/parquet_schema_inspect.txt	TXT	23,0
evidence/query_latency.csv	CSV	1,7
evidence/query_latency.md	MD	2,8
evidence/scale_experiment.csv	CSV	0,3
evidence/scale_experiment.md	MD	1,2
evidence/superset_chart_latency.csv	CSV	2,0
evidence/validation.json	JSON	8,3
evidence/validation.md	MD	3,1
evidence/watchlist_goi_y_quyet_dinh.png	Ảnh	62,0
evidence/watchlist_seller_canh_bao.png	Ảnh	110,0

Phụ lục F. Mã nguồn và cấu hình

- Kho mã nguồn của nhóm: https://github.com/viettuong287/BigDataProject
- Đầu vào upstream: repo `Analytics-main` do nhóm Analytics cung cấp (tài liệu bàn giao sao chép trong `docs/upstream/`); repo Processing: https://github.com/NguyenAlpha/big-data
- `docker-compose.yml` gồm các dịch vụ `superset-db` (Postgres 16, metadata), `superset` (4.1.2, cổng 8088, 4 worker), `tools` (Python 3.12: loader, kiểm chứng, dựng dashboard, đo đạc) và `screenshots` (Playwright).

```
BigDataProject/
├── data/            (Parquet nhận từ Analytics: analytics/, gold/)
├── docs/
│   ├── architecture.md
│   ├── architecture.png
│   └── architecture_pipeline_position.png
├── evidence/
├── loader/
│   ├── apply_sql.py
│   ├── common.py
│   ├── Dockerfile
│   ├── load_to_neon.py
│   ├── requirements.txt
│   └── validate.py
├── report/
├── scripts/
│   ├── Dockerfile.screenshots
│   ├── inspect_parquet.py
│   ├── measure_latency.py
│   ├── neon_inventory.py
│   ├── render_diagrams.py
│   ├── scale_experiment.py
│   └── take_screenshots.py
├── sql/
│   ├── 10_presentation_views.sql
│   └── 90_anchor_checks.sql
├── superset/
│   ├── export/
│   ├── build_dashboard.py
│   ├── docker-init.sh
│   ├── Dockerfile
│   ├── import_dashboard.py
│   ├── superset_client.py
│   └── superset_config.py
├── .env.example
├── .gitignore
├── docker-compose.yml
└── README.md
```

Tệp `superset/superset_config.py`:

```
"""Cấu hình Superset cho môi trường demo cục bộ (docker compose)."""
import os

SECRET_KEY = os.environ.get("SUPERSET_SECRET_KEY", "change-me-in-.env")
```

```
# Metadata (dashboard, chart, user, log) nằm trong container superset-db (Postgres cục
bộ).
# SQLite chỉ là phương án dự phòng: nó khoá ghi toàn cục nên các chart tải song song
bị xếp hàng.
# Dữ liệu nghiệp vụ nằm ở Neon Postgres, đăng ký qua REST API
(superset/build_dashboard.py).
SQLALCHEMY_DATABASE_URI = os.environ.get(
    "SUPERSET_METADATA_URI",
"sqlite:////app/superset_home/superset.db?check_same_thread=false"
)

FEATURE_FLAGS = {
    "DASHBOARD_NATIVE_FILTERS": True,
    "DASHBOARD_CROSS_FILTERS": True,
    "ENABLE_TEMPLATE_PROCESSING": True,
}

# Cache kết quả truy vấn chart 10 phút: lần mở dashboard thứ 2 không phải hỏi lại Neon.
DATA_CACHE_CONFIG = {
    "CACHE_TYPE": "FileSystemCache",
    "CACHE_DIR": "/app/superset_home/cache",
    "CACHE_DEFAULT_TIMEOUT": 600,
}

TALISMAN_ENABLED = False
WTF_CSRF_TIME_LIMIT = None
ROW_LIMIT = 50000
BABEL_DEFAULT_LOCALE = "en"
```

Biến môi trường (`.env.example`, tệp `.env` thật không được commit): `NEON_DATABASE_URL` (endpoint pooled, `sslmode=require`), `NEON_PROJECT_ID`, `SUPERSET_SECRET_KEY`, `SUPERSET_ADMIN_USER`, `SUPERSET_ADMIN_PASSWORD`.

Phụ lục G. Bảng phân công và đóng góp

Bảng G.1. Bảng phân công nhiệm vụ và mức đóng góp của các thành viên

STT	Họ và tên	MSSV	Vai trò	Nhiệm vụ thực tế	Đóng góp
1	Võ Việt Tường	3123411334	Nhóm trưởng – kiến trúc, Neon & loader, tích hợp	Thiết kế kiến trúc và ranh giới với nhóm Analytics/Decision; Khởi tạo project Neon, schema analytics/gold/presentation; Viết loader Parquet → Neon (COPY, khoá chính, load_log) và script kiểm chứng; Docker Compose, tích hợp toàn hệ thống, quản lý repo GitHub	33%
2	Nguyễn Hoàng Phi Long	3123411180	Semantic layer SQL & dashboard Superset	Thiết kế tầng ngữ nghĩa: 22 view, dim_state, định nghĩa KPI; Luật gợi ý quyết định (v_decision_hints, v_seller_watchlist); Dựng dashboard 7 tab, 45 chart bằng REST API; cấu hình 7 bộ lọc native	33%
3	Mai Hồng Phong	3123411225	Thực nghiệm, minh chứng & báo cáo	Đo độ trễ truy vấn, thời gian tải dashboard, thí nghiệm mở rộng ×10; Chụp ảnh minh chứng, kiểm thử bộ lọc, kiểm kê Neon; Viết báo cáo, tài liệu hướng dẫn chạy, tổng hợp tài liệu tham khảo	34%

Bảng G.2. Ma trận đóng góp theo sản phẩm bàn giao (C = chính, H = hỗ trợ)

Sản phẩm	Võ Việt Tường	Nguyễn Hoàng Phi Long	Mai Hồng Phong
Kiến trúc, ranh giới, data contract	C	H	H
Project Neon, loader, `load_log`, `validate.py`	C	H	
Tầng ngữ nghĩa: 22 view, KPI, luật quyết định	H	C	
Superset Docker, dashboard 7 tab, 7 bộ lọc	H	C	H
Đo độ trễ, tải dashboard, thí nghiệm ×10	H		C
Ảnh minh chứng, kiểm thử bộ lọc, kiểm kê Neon		H	C
Báo cáo, README, tài liệu tham khảo	H	H	C
Tích hợp Docker Compose, GitHub	C		H
Mức đóng góp	**33%**	**33%**	**34%**

Phụ lục H. Ghi chú sử dụng công cụ AI

Theo Mục 8 của yêu cầu đồ án, nhóm xin cam kết như sau. Trong quá trình thực hiện, nhóm có sử dụng trợ lý lập trình AI (các agent trong trình soạn thảo Cursor) để hỗ trợ: gợi ý khung mã cho loader, script dựng dashboard qua REST API, script đo đạc và Dockerfile; gợi ý câu lệnh SQL cho tầng ngữ nghĩa; dựng script sinh báo cáo và bản nháp nội dung báo cáo từ tài liệu bàn giao và các tệp minh chứng.

Nhóm đã tự chạy toàn bộ hệ thống, đọc và chỉnh sửa mã, kiểm tra lại số liệu: mọi con số trong báo cáo được lấy trực tiếp từ tệp Parquet, log và tệp đo trong `evidence/` (không do AI tự tạo), được đối chiếu với 19 số neo của nhóm Analytics và với ảnh chụp dashboard. Các quyết định kỹ thuật — chọn Neon và Superset, thiết kế schema, định nghĩa KPI, luật gợi ý quyết định và tiêu chí đánh giá — do nhóm thảo luận và có lập luận ở Chương 2, 4 và 6. Nhóm chịu trách nhiệm hoàn toàn về nội dung nộp và có thể giải thích từng thành phần của hệ thống.